

Team Notebook

Pontificia Universidad Católica de Chile - MEMOir del pudu

1 Data Structures	2	3 Geometry	11	6.2 digitdp	21
1.1 sparsetable	2	3.1 convexhull	11	6.3 linecontainer	21
1.2 segment tree lazy	2	3.2 minkowski	12	6.4 1d1d	21
1.3 dynamic line segment tree	2	3.3 circle line intersection	12	6.5 broken profile	22
1.4 segment tree	3	3.4 circlehierarchy	12	6.6 sos dp	22
1.5 ordered set	3	3.5 closest points	13	6.7 dp divide and conquer	22
1.6 tuples unordered map	3	3.6 point in polygon	13	7 Strings	23
1.7 dynamic lazy segment tree	3	3.7 segment	13	7.1 hashing2d	23
1.8 ordered set	4	3.8 halfplane intersection	13	7.2 aho corasick	23
1.9 merge sort tree	4	3.9 point	14	7.3 hashing	23
1.10 mo	5	3.10 polygon	14	7.4 kmp	24
1.11 segmenttree2d	5	3.11 delaunay	14	7.5 manacher	24
1.12 persistent segment tree	5	4 Algorithms	15	7.6 suffix array	24
1.13 line segment tree	6	4.1 ternary search	15	7.7 trie	24
2 Math	6	5 Graphs	16	7.8 suffix automaton	25
2.1 linear diophantine	6	5.1 mincostmaxflow	16	7.9 palindromic tree	25
2.2 mobius	6	5.2 chuliu	16	7.10 rolling hash	25
2.3 128bits	6	5.3 scc	16		
2.4 gcd lcm	7	5.4 dsu	17		
2.5 fft	7	5.5 dominatortree	17		
2.6 theorems	7	5.6 bridges	17		
2.7 matrix	8	5.7 hungarian	18		
2.8 crt	8	5.8 floydwarshall	18		
2.9 system mod 2	8	5.9 2sat	18		
2.10 theorems and formulas	9	5.10 partiallypersistentdsu	19		
2.11 simplex	9	5.11 cut points	19		
2.12 combinatorics	10	5.12 dinic	19		
2.13 fht	10	5.13 specialpathwithcentroids	20		
2.14 sieve	10	5.14 heavylightdecomposition	20		
2.15 gauss mod 2	11	6 Dp	21		
2.16 gauss	11	6.1 knuth	21		

1 Data Structures

1.1 sparsetable

```
struct SparseTable{
    vector<vl> >SP;
    SparseTable(vl&A){
        int n = A.size();
        SP.push_back(A);
        ll maxlog = 31 - __builtin_clz(n);
        repx(i, 1, maxlog+1){
            vl aux;
            rep(j, n-(1<<i)+1){
                aux.push_back(min(SP[i-1][j], SP[i-1]
[j+(1<<(i-1))]]));
            }
            SP.push_back(aux);
        }
        ll op(int l, int r){
            ll maxlog = 31 - __builtin_clz(r-l+1);
            return max(SP[maxlog][l], SP[maxlog][r-
(1<<maxlog)+1]);
        }
        ll find(int l, int r, ll m){ // maxi
            ll maxlog = 31 - __builtin_clz(r-l+1);
            for(int i = maxlog; i >= 0; i--){
                if(l + (1<<i) <= r && SP[i][l] < m){
                    l += (1<<i);
                }
            }
            return l;
        }
    };
};
```

1.2 segment tree lazy

```
template <class T1, class T2, T1 merge(T1, T1),
void pushUpdate(T2 parent, T2 &child, int, int,
int, int),
void applyUpdate(T2 update, T1 &node, int, int)>
struct SegmentTreeLazy {
    int n;
    vector<T1> tree;
    vector<T2> lazy;
    vector<bool> isUpdated;

    void build(int i, int left, int right, const vector<T1>
&values) {
        if (left == right) {
            tree[i] = values[left];
            return;
        }
    }
```

```
int mid = (left + right) >> 1;

build(i << 1, left, mid, values);
build(i << 1 | 1, mid + 1, right, values);

tree[i] = merge(tree[i << 1], tree[i << 1 | 1]);
}

SegmentTreeLazy(const vector<T1> &values) {
    n = values.size();
    int size = n << 2 | 3;

    tree.resize(size);
    lazy.resize(size);
    isUpdated.resize(size);

    build(1, 0, n - 1, values);
}

SegmentTreeLazy() {}

void push(int i, int left, int right) {
    if (!isUpdated[i])
        return;

    applyUpdate(lazy[i], tree[i], left, right);

    if (left != right) {
        if (isUpdated[i << 1])
            pushUpdate(lazy[i], lazy[i << 1], left, right, left,
(left + right) / 2);

        else
            lazy[i << 1] = lazy[i];

        if (isUpdated[i << 1 | 1])
            pushUpdate(lazy[i], lazy[i << 1 | 1], left, right,
(left + right) / 2 + 1, right);

        else
            lazy[i << 1 | 1] = lazy[i];

        isUpdated[i << 1] = 1;
        isUpdated[i << 1 | 1] = 1;
    }

    isUpdated[i] = false;
}

void update(int i, int left, int right, int qLeft, int
qRight, T2 &value) {
    if (left >= qLeft and right <= qRight) {
        if (isUpdated[i])
            pushUpdate(value, lazy[i], qLeft, qRight, left,
right);
        else
            lazy[i] = value;
    }
```

```
isUpdated[i] = true;
}

push(i, left, right);

if (left > qRight or right < qLeft)
    return;

if (left >= qLeft and right <= qRight)
    return;

update(i << 1, left, (left + right) >> 1, qLeft, qRight,
value);
update(i << 1 | 1, (left + right) / 2 + 1, right, qLeft,
qRight, value);

tree[i] = merge(tree[i << 1], tree[i << 1 | 1]);
}

void update(int left, int right, T2 value) {
    update(1, 0, n - 1, left, right, value);
}

T1 query(int i, int left, int right, int qLeft, int
qRight) {
    push(i, left, right);

    if (qLeft <= left and right <= qRight)
        return tree[i];

    int mid = (left + right) >> 1;

    if (mid < qLeft)
        return query(i << 1 | 1, mid + 1, right, qLeft,
qRight);

    if (mid >= qRight)
        return query(i << 1, left, mid, qLeft, qRight);

    return merge(query(i << 1, left, mid, qLeft, qRight),
query(i << 1 | 1, mid + 1, right, qLeft,
qRight));
}

T1 query(int left, int right) { return query(1, 0, n - 1,
left, right); }
```

1.3 dynamic line segment tree

```
struct DynamicLineSegmentTree {
    vector<LineContainer> tree;
    vector<int> L, R;
```

```

ll minX, maxX;

DynamicLineSegmentTree(ll left, ll right) {
    tree.resize(1);
    L = {0};
    R = {0};
    minX = left;
    maxX = right;
}

int newNode() {
    int i = tree.size();
    tree.push_back(LineContainer());
    L.push_back(0);
    R.push_back(0);
    return i;
}

void add(int i, ll left, ll right, ll queryLeft, ll
queryRight, ll a, ll b) {
    if (left > queryRight or right < queryLeft)
        return;

    if (queryLeft <= left && right <= queryRight) {
        tree[i].add(a, b);
        return;
    }

    int mid = (left + right) >> 1;

    if (L[i] == 0)
        L[i] = newNode();

    if (R[i] == 0)
        R[i] = newNode();

    add(L[i], left, mid, queryLeft, queryRight, a, b);
    add(R[i], mid + 1, right, queryLeft, queryRight, a, b);
}

void add(int left, int right, ll a, ll b) {
    add(0, minX, maxX, left, right, a, b);
}

ll query(int i, int left, int right, ll x) {
    ll answer = tree[i].query(x);

    if (left == right)
        return answer;

    int mid = (left + right) >> 1;

    // change max to min for min
    if (x <= mid && L[i] != 0)
        answer = max(answer, query(L[i], left, mid, x));

```

```

    if (x > mid && R[i] != 0)
        answer = max(answer, query(R[i], mid + 1, right, x));

    return answer;
}

ll query(ll x) { return query(0, minX, maxX, x); }
};

```

1.4 segment tree

```

template <class T, T merge(T, T)> struct SegmentTree {
    int n;
    vector<T> tree;
    SegmentTree() {}
    SegmentTree(vector<T> &values) {
        n = values.size();
        tree.resize(n << 1);
        for (int i = 0; i < n; i++)
            tree[i + n] = values[i];
        for (int i = n - 1; i > 0; i--)
            tree[i] = merge(tree[i << 1], tree[i << 1 | 1]);
    }
    void update(int position, T value) {
        position += n;
        tree[position] = value;
        for (position >= 1; position > 0; position >= 1)
            tree[position] = merge(tree[position << 1],
tree[position << 1 | 1]);
    }
    T query(int left,
            int right) // [left, right]
    {
        bool hasAnswer = false;
        T answer = T();
        for (left += n, right += n + 1; left < right; left >=
1, right >= 1) {
            if (left & 1) {
                answer = hasAnswer ? merge(answer, tree[left]) :
tree[left];
                hasAnswer = true;
                left++;
            }
            if (right & 1) {
                right--;
                answer = hasAnswer ? merge(answer, tree[right]) :
tree[right];
                hasAnswer = true;
            }
        }
        return answer;
    }
};

```

1.5 ordered set

```

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

template <typename T>
using ordered_set =
    tree<T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update>;

template <typename T>
using ordered_multiset = tree<T, null_type, less_equal<T>,
rb_tree_tag,
tree_order_statistics_node_update>;

template <typename T> void erase(ordered_multiset<T>
&values, T value) {
    int rank = values.order_of_key(value);
    auto it = values.find_by_order(rank);
    values.erase(it);
}

```

1.6 tuples unordered map

```

using key = tuple<int, int, int>; // define a key type

using bits = bitset<sizeof(key) * 8>;
unordered_map<bits, int> hashmap;

// hashmap[*((long long *)&hash)] = value;

```

1.7 dynamic lazy segment tree

```

// Not well tested
// Not works for too large n like 10^18
// Use when n is large but number of updates/queries is
small

template <class Node, // Value Type
        class Lazy, // Lazy type
        class Index, // Range type
        Node (*merge)(Node, Node),
        void (*pushUpdate)(Lazy &parent, Lazy &child,
Index a, Index b,
                        Index l, Index r),
        void (*applyUpdate)(Lazy &update, Node &values,
Index l, Index r),
        Node (*createNewNode)(Index l, Index r), //
Create default value
        Node (*createNeutralNode)(Index l, Index r)> //
Create neutral value
struct DynamicLazySegmentTree {
    vector<Node> values;

```

```

vector<Lazy> lazy;
vector<bool> hasUpdate;
vector<int> L, R;
int root, nodeCount;
Index n;

DynamicLazySegmentTree(Index maxIndex, int size = 1 << 20)
: n(maxIndex), nodeCount(1) {
    values.reserve(size);
    lazy.reserve(size);
    hasUpdate.reserve(size);
    L.reserve(size);
    R.reserve(size);

    values.push_back(createNewNode(0, n - 1));
    lazy.push_back(Lazy());
    hasUpdate.push_back(false);
    L.push_back(0);
    R.push_back(0);
    root = 0;
}

int createNode(Index l, Index r) {
    int i = nodeCount++;
    values.push_back(createNewNode(l, r));
    lazy.push_back(Lazy());
    hasUpdate.push_back(false);
    L.push_back(0);
    R.push_back(0);
    return i;
}

void push(int i, Index l, Index r) {
    if (!hasUpdate[i])
        return;

    if (l != r) {
        Index mid = (l + r) >> 1;

        if (!L[i])
            L[i] = createNode(l, mid);

        if (!R[i])
            R[i] = createNode(mid + 1, r);

        pushUpdate(lazy[i], lazy[L[i]], l, r, l, mid);
        pushUpdate(lazy[i], lazy[R[i]], l, r, mid + 1, r);

        hasUpdate[L[i]] = true;
        hasUpdate[R[i]] = true;

        applyUpdate(lazy[i], values[L[i]], l, mid);
        applyUpdate(lazy[i], values[R[i]], mid + 1, r);
    }
}

```

```

lazy[i] = Lazy();
hasUpdate[i] = false;
}

void update(int i, Index l, Index r, Index a, Index b,
Lazy &value) {
    if (r < a || l > b)
        return;

    if (a <= l && r <= b) {
        applyUpdate(value, values[i], l, r);

        if (l != r)
            if (hasUpdate[i])
                applyUpdate(lazy[i], values[i], l, r);
            else {
                lazy[i] = value;
                hasUpdate[i] = true;
            }

        return;
    }

    push(i, l, r);

    Index mid = (l + r) >> 1;

    if (!L[i])
        L[i] = createNode(l, mid);

    update(L[i], l, mid, a, b, value);

    if (!R[i])
        R[i] = createNode(mid + 1, r);

    update(R[i], mid + 1, r, a, b, value);

    values[i] = merge(values[L[i]], values[R[i]]);
}

void update(Index a, Index b, Lazy value) {
    update(root, 0, n - 1, a, b, value);
}

void update(Index i, Lazy value) { update(i, i, value); }

Node query(int i, Index l, Index r, Index a, Index b) {
    push(i, l, r);

    if (r < a || l > b)
        return createNeutralNode(l, r);

    if (a <= l && r <= b)
        return values[i];
}

```

```

Index mid = (l + r) >> 1;

Node left = createNewNode(l, mid);
Node right = createNewNode(mid + 1, r);

if (L[i] != 0)
    left = query(L[i], l, mid, a, b);

if (R[i] != 0)
    right = query(R[i], mid + 1, r, a, b);

return merge(left, right);
}

Node query(Index a, Index b) { return query(root, 0, n - 1, a, b); }

Node query(Index i) { return query(i, i); }
};

```

1.8 ordered set

```

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

typedef tree<int, null_type, less<int>, rb_tree_tag,
            tree_order_statistics_node_update> ordered_set;
// find_by_order(i) -> iterator to ith element
// order_of_key(k) -> position (int) of lower_bound of k

```

1.9 merge sort tree

```

struct MergeSortTree {
    int n;
    vector<vector<int>> tree;

    vector<int> merge(const vector<int> &left, const
vector<int> &right) {
        vector<int> result(left.size() + right.size());

        int i = 0, j = 0;

        for (int &x : result) {
            if (i == left.size())
                x = right[j++];
            else if (j == right.size())
                x = left[i++];
            else if (left[i] < right[j])
                x = left[i++];
            else
                x = right[j++];
        }

        return result;
    }
};

```

```

}

void build(int i, int left, int right, const vector<int>
&values) {
    if (left == right) {
        tree[i] = {values[left]};
        return;
    }

    int mid = (left + right) >> 1;

    build(i << 1, left, mid, values);
    build(i << 1 | 1, mid + 1, right, values);

    tree[i] = merge(tree[i << 1], tree[i << 1 | 1]);
}

MergeSortTree(const vector<int> &values) {
    n = values.size();
    tree.resize(n << 2 | 3);
    build(1, 0, n - 1, values);
}

int query(int i, int left, int right, int queryLeft, int
queryRight, int min,
          int max) {
    if (left > queryRight or right < queryLeft)
        return 0;

    if (left >= queryLeft and right <= queryRight) {
        auto leftIterator = lower_bound(tree[i].begin(),
tree[i].end(), min);
        auto rightIterator = upper_bound(tree[i].begin(),
tree[i].end(), max);

        return distance(leftIterator, rightIterator);
    }

    int mid = (left + right) >> 1;

    return query(i << 1, left, mid, queryLeft, queryRight,
min, max) +
           query(i << 1 | 1, mid + 1, right, queryLeft,
queryRight, min, max);
}

int query(int left, int right, int min, int max) {
    return query(1, 0, n - 1, left, right, min, max);
}
};

```

1.10 mo

```
struct Query { int l, r, idx; };
```

```

// answer segment queries using only `add(i)`, `remove(i)`
// and `get()`
// functions.
//
// nota marcelo: borra el add, remove y get de la funcion mo
// y
// definelo afuera
//
// complexity: O((N + Q) * sqrt(N) * F)
//   N = length of the full segment
//   Q = amount of queries
//   F = complexity of the `add`, `remove` functions
template <class A, class R, class G, class T>
void mo(vector<Query> &queries, vector<T> &ans, A add, R
remove, G get) {
    int Q = queries.size(), B = (int)sqrt(Q);
    sort(queries.begin(), queries.end(), [&](Query &a, Query
&b) {
        if (p.first / BLOCK_SIZE != q.first / BLOCK_SIZE)
            return p < q;
        return (p.first / BLOCK_SIZE & 1) ? (p.second <
q.second) : (p.second > q.second);
    });
    ans.resize(Q);

    int l = 0, r = 0;
    for (auto &q : queries) {
        while (r < q.r) add(r), r++;
        while (l > q.l) l--, add(l);
        while (r > q.r) r--, remove(r);
        while (l < q.l) remove(l), l++;
        ans[q.idx] = get();
    }
}

```

1.11 segmenttree2d

```

struct SegmentTree{
    vector<ll>ST; int N;
    SegmentTree(int Size){
        N = Size; ST.assign(4*N,0);
    }
    void upd(int i, ll v){
        return upd(1,0,N-1,i,v);
    }
    void upd(int n, int l, int r, int i, ll v){
        if(i < l || r < i) return;
        if(l == r){
            ST[n] += v; return;
        }
        upd(2*n,l,(l+r)/2,i,v);
        upd(2*n+1,(l+r)/2+1,r,i,v);
        ST[n] = ST[2*n]+ST[2*n+1];
    }
    ll qry(int i,int j){

```

```

        return qry(1,0,N-1,i,j);
    }
    ll qry(int n, int l, int r, int i, int j){
        if(r < i || j < l) return 0;
        if(i <= l && r <= j) return ST[n];
        return query(2*n,l,(l+r)/2,i,j)+query(2*n+1,
(l+r)/2+1,r,i,j);
    }
};

struct SegmenTree2D{
    vector<SegmentTree>ST;
    int N;
    SegmenTree2D(int Size){
        N = Size; ST.resize(4*N,Size);
    }
    void update(int i, int j, int v){
        return update(1,0,N-1,i,j,v);
    }
    void update(int n, int l, int r, int i, int j, ll v){
        if(i < l || r < i) return;
        if(l == r){
            ST[n].update(j,v);
            return;
        }
        update(2*n,l,(l+r)/2,i,j,v);
        update(2*n+1,(l+r)/2+1,r,i,j,v);
        ST[n].update(j,v);
    }
    ll query(int i1, int i2, int j1, int j2){
        return query(1,0,N-1,i1,i2,j1,j2);
    }
    ll query(int n, int l, int r, int i1, int i2, int j1,
int j2){
        if(l > j1 || i1 > r) return 0;
        if(i1 <= l && r <= j1){
            return ST[n].query(i2,j2);
        }
        return query(2*n,l,(l+r)/2,i1,i2,j1,j2)+query(2*n+1,
(l+r)/2+1,r,i1,i2,j1,j2);
    }
};

```

1.12 persistent segment tree

```

template <class T, T merge(T, T)> struct
PersistentSegmentTree {
    vector<T> tree;
    vector<int> L, R;
    int n, root = 0;

    PersistentSegmentTree(int n) : n(n) {
        tree.resize(1);
        L = {0};
        R = {0};
    }
};

```

```

int newNode(T value, int left = 0, int right = 0) {
    int id = tree.size();
    tree.push_back(value);
    L.push_back(left);
    R.push_back(right);
    return id;
}

int update(int id, int left, int right, int position, T
value) {
    int newId = newNode(tree[id], L[id], R[id]);

    if (left == right) {
        tree[newId] = value;
        return newId;
    }

    int mid = (left + right) >> 1;

    if (position <= mid)
        L[newId] = update(L[newId], left, mid, position,
value);
    else
        R[newId] = update(R[newId], mid + 1, right, position,
value);

    tree[newId] = merge(tree[L[newId]], tree[R[newId]]);

    return newId;
}

T query(int id, int left, int right, int qLeft, int
qRight) {
    if (left >= qLeft and right <= qRight)
        return tree[id];

    int mid = (left + right) >> 1;

    if (qRight <= mid)
        return query(L[id], left, mid, qLeft, qRight);

    if (qLeft > mid)
        return query(R[id], mid + 1, right, qLeft, qRight);

    return merge(query(L[id], left, mid, qLeft, qRight),
        query(R[id], mid + 1, right, qLeft,
qRight));
}

T query(int id, int left, int right) {
    return query(id, 0, n - 1, left, right);
}

int update(int id, int position, T value) {
    return root = update(id, 0, n - 1, position, value);
}

```

```

}

int update(int position, T value) { return update(root,
position, value); }
};

```

1.13 line segment tree

```

struct LineSegmentTree {
    int n;
    vector<LineContainer> tree;

    LineSegmentTree(int n) : n(n) { tree.resize(4 * n); }

    void add(int i, int left, int right, int queryLeft, int
queryRight, ll a,
            ll b) {
        if (left > queryRight or right < queryLeft)
            return;

        if (queryLeft <= left && right <= queryRight) {
            tree[i].add(a, b);
            return;
        }

        int mid = (left + right) >> 1;

        add(i << 1, left, mid, queryLeft, queryRight, a, b);
        add(i << 1 | 1, mid + 1, right, queryLeft, queryRight,
a, b);
    }

    void add(int left, int right, ll a, ll b) {
        add(1, 0, n - 1, left, right, a, b);
    }

    ll query(int i, int left, int right, ll x) {
        ll answer = tree[i].query(x);

        if (left == right)
            return answer;

        int mid = (left + right) >> 1;

        // change max to min for min
        if (x <= mid)
            answer = max(answer, query(i << 1, left, mid, x));
        else
            answer = max(answer, query(i << 1 | 1, mid + 1, right,
x));

        return answer;
    }
}

```

```

ll query(ll x) { return query(1, 0, n - 1, x); }
};

```

2 Math

2.1 linear diophantine

```

ii extendedEuclid(ll a, ll b){
    ll x, y; //a*x + b*y = gcd(a,b)
    if (b == 0) return {1, 0};
    auto p = extendedEuclid(b, a%b);
    x = p.second;
    y = p.first - (a/b)*x;
    if(a*x + b*y == -__gcd(a,b)) x=-x, y=-y;
    return {x, y};
}

pair<ii, ii> diophantine(ll a, ll b, ll r){
    //a*x+b*y=r where r is multiple of gcd(a,b);
    ll d = __gcd(a, b);
    a/=d; b/=d; r/=d;
    auto p = extendedEuclid(a, b);
    p.first*=r; p.second*=r;
    assert(a*p.first + b*p.second == r);
    return {p, {-b, a}}; //solutions: p+t*ans.second
}

```

2.2 mobius

```

short mu[MAXN] = {0,1};
void mobius(){
    repx(i,1,MAXN)if(mu[i])for(int j=i+i;j<MAXN;j+=i)mu[j]-
=mu[i];
}

```

2.3 128bits

```

#if defined(__LP64__) || defined(_WIN64)
using int128 = __int128;
using uint128 = __uint128_t;
#else
using int128 = ll;
using uint128 = ull;
#endif

template <typename T>
typename enable_if<is_same<T, int128>::value || is_same<T,
uint128>::value,
                istream &>::type
operator>>(istream &is, T &value) {
    string input;
    is >> input;

    value = 0;
}

```

```

for (char character : input)
    if (isdigit(character))
        value = 10 * value + character - '0';

if (input[0] == '-')
    value *= -1;

return is;
}

template <typename T>
typename enable_if<is_same<T, int128>::value || is_same<T,
uint128>::value,
                ostream &>::type
operator<<(ostream &os, T value) {
    string output;

    bool isNegative = value < 0;

    if (isNegative) {
        value *= -1;
        output += '-';
    }

    do {
        output += value % 10 + '0';
        value /= 10;
    } while (value > 0);

    reverse(output.begin() + isNegative, output.end());

    return os << output;
}

```

2.4 gcd lcm

```

#define gcd __gcd

template <typename T>
typename enable_if<is_integral<T>::value, T>::type lcm(T a,
T b) {
    return a / gcd(a, b) * b;
}

```

2.5 fft

```

#define M_PIL 3.141592653589793238462643383279502884L

typedef complex<double> C;
typedef vector<double> vd;

void fft(vector<C> &a)
{
    int n = a.size(), L = 31 - __builtin_clz(n);
    static vector<complex<long double>> R(2, 1);

```

```

static vector<C> rt(2, 1);
for (static int k = 2; k < n; k *= 2)
{
    R.resize(n);
    rt.resize(n);
    auto x = polar(1.0L, M_PIL / k);
    repx(i, k, 2 * k) rt[i] = R[i] = i & 1 ? R[i / 2] *
x : R[i / 2];
}
vector<int> rev(n);
rep(i, n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
rep(i, n) if (i < rev[i]) swap(a[i], a[rev[i]]);
for (int k = 1; k < n; k *= 2) for (int i = 0; i < n; i
+= 2 * k) rep(j, k)
{
    auto x = (double *)&rt[j + k], y = (double *)&a[i +
j + k];
    C z(x[0] * y[0] - x[1] * y[1], x[0] * y[1] + x[1] *
y[0]);
    a[i + j + k] = a[i + j] - z;
    a[i + j] += z;
}

vd conv(const vl &a, const vl &b)
{
    if (a.empty() || b.empty()) return {};
    vd res(a.size() + b.size() - 1);
    int L = 32 - __builtin_clz(res.size()), n = 1 << L;
    vector<C> in(n), out(n);
    copy(a.begin(), a.end(), in.begin());
    rep(i, b.size()) in[i].imag(b[i]);
    fft(in);
    for (auto &x : in) x *= x;
    rep(i, n) out[i] = in[-i & (n - 1)] - conj(in[i]);
    fft(out);
    rep(i, res.size()) res[i] = imag(out[i]) / (4 * n);
    return res;
}

//slower
vl convMod(const vl &a, const vl &b, int M)
{
    if (a.empty() || b.empty()) return {};
    vl res(a.size() + b.size() - 1);
    int B = 32 - __builtin_clz(res.size()), n = 1 << B, cut
= int(sqrt(M));
    vector<C> L(n), R(n), outs(n), outl(n);
    rep(i, a.size()) L[i] = C((int)a[i] / cut, (int)a[i] %
cut);
    rep(i, b.size()) R[i] = C((int)b[i] / cut, (int)b[i] %
cut);
    fft(L), fft(R);
    rep(i, n)
{

```

```

int j = -i & (n - 1);
outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) /
li;
}
fft(outl), fft(outs);
rep(i, res.size())
{
    ll av = ll(real(outl[i]) + .5), cv =
ll(imag(outs[i]) + .5);
    ll bv = ll(imag(outl[i]) + .5) + ll(real(outs[i])
+ .5);
    res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
}
return res;
}

```

2.6 theorems

Burnside lemma

Tomemos imagenes x en X y operaciones $(g: X \rightarrow X)$ en G . Si $\#g$ es la cantidad de imagenes que son puntos fijos de g , entonces la cantidad de objetos es $\frac{\sum_{g \in G} \#g}{|G|}$. Es requisito que G tenga la operacion identidad, que toda operacion tenga inversa y que todo par de operaciones tenga su combinacion.

Rational root theorem

Las raices racionales de un polinomio de orden n con coeficientes enteros $A[i]$ son de la forma p/q , donde p y q son coprimos, p es divisor de $A[0]$ y q es divisor de $A[n]$. Notar que si $A[0] = 0$, cero es raiz, se puede dividir el polinomio por x y aplica nuevamente el teorema.

Petersens theorem

Every cubic and bridgeless graph has a perfect matching.

Number of divisors for powers of 10

(0,1) (1,4) (2,12) (3,32) (4,64) (5,128) (6,240) (7,448) (8,768) (9,1344) (10,2304) (11,4032) (12,6720) (13,10752) (14,17280) (15,26880) (16,41472) (17,64512) (18,103680)

Kirchoff Theorem: Sea A la matriz de adyacencia del multi-grafo ($A[u][v]$ indica la cantidad de aristas entre u y v) Sea D una matriz diagonal tal que $D[v][v]$ es igual al grado de v (considerando auto aristas y multi aristas). Sea $L = A - D$. Todos los cofactores de L son iguales y equivalen a la cantidad de Spanning Trees del grafo. Un cofactor (i,j) de L es la multiplicación de $(-1)^{i+j}$ con el determinant de la matriz al quitar la fila i y la columna j

Prufer Code: Dado un árbol con los nodos indexados: busca la hoja de menor índice, bórrala y anota el índice del nodo al que estaba conectado. Repite el paso anterior $n-2$ veces. Lo anterior muestra una biyección entre los arreglos de tamaño

$n-2$ con elementos en $[1, n]$ y los árboles de n nodos, por lo que hay n^{n-2} spanning trees en un grafo completo.
 Corolario: Si tenemos k componentes de tamaños $s_1, s_2, \dots, s_k$ entonces podemos hacerlos conexos agregando $k-1$ aristas entre nodos de $s_1 s_2 \dots s_k n^{k-2}$ formas

Combinatoria

Catalan: $C_{n+1} = \sum (C_i * C_{n-i})$ for $i \in [0, n]$

Catalan: $C_n = \frac{1}{n+1} \binom{2n}{n}$

Sea C_n^k las formas de poner $n+k$ pares de paréntesis, con los primeros k paréntesis abiertos (esto es, hay $2n + 2k$ caracteres), se tiene que

$C_n^k = (2n+k-1) * (2n+k) / (n * (n+k+1)) * C_{n-1}^k$

Sea D_n el número de permutaciones sin puntos fijos, entonces $D_n = (n-1)(D_{n-1} + D_{n-2})$, $D_0 = 1$, $D_1 = 0$

2.7 matrix

```
#define mat vector<vector<ll>>

const int M = 1e9+7;
const ll M2 = (ll)M*M;

mat cn(int n, int m){
    return vector<vector<ll>>(n, vector<ll>(m));
}

mat mult(mat &a, mat &b){
    mat c = cn(a.size(), b[0].size());
    for(int i=0; i<a.size(); ++i){
        for(int k = 0; k<b.size(); ++k){
            for(int j = 0; j<b[0].size(); ++j){
                c[i][j] += a[i][k] * b[k][j];
                if(c[i][j] >= M2){
                    c[i][j] -= M2;
                }
            }
        }
    }
    for(int i = 0; i<a.size(); ++i){
        for(int j=0; j<b[0].size(); ++j){
            c[i][j] %= M;
        }
    }
    return c;
}
```

2.8 crt

```
pair<ll, ll> solve_crt(const vector<pair<ll, ll>> &eqs) {
    ll a0 = eqs[0].first, p0 = eqs[0].second;
    rep(x, 1, eqs.size()) {
        ll a1 = eqs[x].first, p1 = eqs[x].second;
        ll k1, k0;
        ll d = ext_gcd(p1, p0, k1, k0);
```

```
    a0 -= a1;
    if (a0 % d != 0) return {-1, -1};
    p0 = p0 / d * p1;
    a0 = a0 / d * k1 % p0 * p1 % p0 + a1;
    a0 = (a0 % p0 + p0) % p0;
}
return {a0, p0};
}
```

2.9 system mod 2

```
struct Mod2System {
    vector<vector<int>> matrix;
    int rows, columns, variables;
    vector<int> pivotPosition, freeColumns;

    Mod2System(vector<vector<int>> matrixToSolve) {
        matrix = matrixToSolve;
        rows = matrix.size();
        columns = (rows > 0 ? matrix[0].size() : 0);
        variables = columns - 1;

        pivotPosition = vector<int>(rows, -1);

        int pivotRow = 0;
        for (int pivotColumn = 0; pivotColumn < variables &&
            pivotRow < rows;
            pivotColumn++) {
            int pivotIndex = pivotRow;

            while (pivotIndex < rows && matrix[pivotIndex]
                [pivotColumn] == 0)
                pivotIndex++;

            if (pivotIndex == rows)
                continue;

            swap(matrix[pivotRow], matrix[pivotIndex]);
            pivotPosition[pivotRow] = pivotColumn;

            for (int i = 0; i < rows; i++)
                if (i != pivotRow && (matrix[i][pivotColumn] & 1))
                    for (int j = pivotColumn; j < columns; j++)
                        matrix[i][j] = matrix[i][j] ^ matrix[pivotRow]
                [j];

            pivotRow++;
        }
    }
}
```

```
vector<bool> isPivot(variables);

for (int position : pivotPosition)
    if (position != -1)
        isPivot[position] = true;
```

```
for (int j = 0; j < variables; j++)
    if (!isPivot[j])
        freeColumns.push_back(j);
}

vector<int> findOneSolution() {
    vector<int> solution(variables);

    if (freeColumns.empty())
        return solution;

    solution[freeColumns[0]] = 1;

    for (int i = 0; i < rows; i++) {
        if (pivotPosition[i] == -1)
            continue;

        int position = pivotPosition[i];
        solution[position] = matrix[i][freeColumns[0]];
    }

    return solution;
}

vector<vector<int>> findSolutions(int maxSolutions = 1e5)
{
    if (freeColumns.empty())
        return {vector<int>(variables)};

    vector<vector<int>> solutions;

    for (int freeColumn : freeColumns) {
        vector<int> solution(variables);
        solution[freeColumn] = 1;

        for (int i = 0; i < rows; i++) {
            if (pivotPosition[i] == -1)
                continue;

            int pivotCol = pivotPosition[i];
            solution[pivotCol] = matrix[i][freeColumn];
        }

        solutions.push_back(solution);

        if (solutions.size() == maxSolutions)
            break;
    }

    return solutions;
}

bool isTrivial(vector<int> &solution) {
    return count(solution.begin(), solution.end(), 1) == 0;
}
```



```

bool onlyHasTrivialSolution() {
    if (freeColumns.empty())
        return true;

    vector<int> solution = findOneSolution();

    return isTrivial(solution);
}
};

```

2.10 theorems and formulas

$$n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$$

$$\sum_{i=0}^k \binom{n+i}{i} = \binom{n+k+1}{k}$$

$[n]_k$ = perm of n elements with k cycles

$$\begin{bmatrix} n+1 \\ k \end{bmatrix} = n \begin{bmatrix} n \\ k \end{bmatrix} + \begin{bmatrix} n \\ k-1 \end{bmatrix}$$

$\left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$ = partitions of an n -element set into k parts

$$\left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\} = \frac{1}{k!} \sum_{i=0}^k (-1)^i \binom{k}{i} (k-i)^n$$

$$\left\{ \begin{smallmatrix} n+1 \\ k \end{smallmatrix} \right\} = k \left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\} + \left\{ \begin{smallmatrix} n \\ k-1 \end{smallmatrix} \right\}$$

Integers $d_1 \geq \dots \geq d_n \geq 0$ can be the degree sequence of a finite simple graph on n vertices $\Leftrightarrow d_1 + \dots + d_n$ is even and for every k in $1 \leq k \leq n$

$$\sum_{i=1}^k d_i \leq k(k-1) + \sum_{i=k+1}^n \min(d_i, k)$$

$$a^n = a^{\varphi(m)+n \bmod \varphi(m)} \pmod{m} \text{ if } n > \lg(m)$$

Misere Nim: if $\exists a_i > 1$ then normal nim; else the condition is reversed.

Derangements: Num of permutations of $n =$

0, 1, 2, ... elements without fixed points is 1, 0, 1, 2, 9, 44, 265, 1854, 14833, ... Recurrence: $D_n = (n-1)(D_{n-1} + D_{n-2}) = n * D_{n-1} + (-1)^n$

Collary: number of permutations with exactly k fixed points $\binom{n}{k} D_{n-k}$

Eulerian numbers: $E(n, k)$ is the number of permutations with exactly k descents ($i : \pi_i <$

π_{i+1}), ascents ($\pi_i > \pi_{i+1}$) / excedances ($\pi > i$) / $k+1$ weak excedances ($\pi \geq i$).

$$E_{n,k} = (k+1)E_{n-1,k} + (n-k)E_{n-1,k-1}$$

Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d) g\left(\frac{n}{d}\right)$$

Other useful formulas/forms

$$\sum_{d|n} \mu(d) = [n=1] \text{ (very useful)}$$

$$g(n) = \sum_{1 \leq m \leq n} f\left(\left\lfloor \frac{n}{m} \right\rfloor\right) \Leftrightarrow f(n) = \sum_{1 \leq m \leq n} \mu(m) g\left(\left\lfloor \frac{n}{m} \right\rfloor\right)$$

$$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu\left(\frac{d}{n}\right) g(d)$$

Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1, \quad p(n) = \sum_{k \in \mathbb{Z} \setminus \{0\}} (-1)^{k+1} p\left(n - \frac{k(3k-1)}{2}\right)$$

$$p(n) \approx \frac{0.145}{n} \cdot e^{2.56\sqrt{n}}$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	$\approx 2e5$	$\approx 2e8$

Bell numbers

Total number of partitions of n distinct elements.

$$B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \dots$$

For p prime,

$$B(p^m + n) \equiv mB(n) + B(n+1) \pmod{p}$$

Labeled unrooted trees

on n vertices: n^{n-2}

on k existing trees of size n_i : $n_1 n_2 \dots n_k n^{k-2}$

with degrees d_i : $\frac{(n-2)!}{(d_1-1)! \dots (d_n-1)!}$

Catalan numbers

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1, \quad C_{n+1} = \frac{2(2n+1)}{n+2} C_n, \quad C_{n+1} = \sum C_i C_{n-i}$$

$$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \dots$$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with n pairs of parenthesis, correctly nested.
- binary trees with $n+1$ leaves (0 or 2 children).
- ordered trees with $n+1$ vertices.
- ways a convex polygon with $n+2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

Narayana Numbers

$$N(n, k) = \frac{1}{n} \binom{n}{k} \binom{n}{k-1}$$

2.11 simplex

```

/* Solves a general linear maximization problem: maximize
$c^T x$ subject to $Ax \le b$, $x \ge 0$. Returns -inf if
there is no solution, inf if there are arbitrarily good
solutions, or the maximum value of $c^T x$ otherwise. The
input vector is set to an optimal $x$ (or in the unbounded
case, an arbitrary solution fulfilling the constraints).
Numerical stability is not guaranteed. For better
performance, define variables such that $x = 0$ is viable.
Usage:
vvd A = {{1,-1}, {-1,1}, {-1,-2}};
vd b = {1,1,-4}, c = {-1,-1}, x;
T val = LPSolver(A, b, c).solve(x);
Time: 0(NM * \#pivots), where a pivot may be e.g. an edge

```

```

relaxation. 0(2^n) in the general case.*/

typedef double T;//long double, Rational, double + mod<P>...
typedef vector<T> vd;
typedef vector<vd> vvd;
const T eps = 1e-8, inf = 1 / .0;
#define MP make_pair
#define ltj(X) \
    if (s == -1 || MP(X[j], N[j]) < MP(X[s], N[s])) s = j
struct LPSolver {
    int m, n; vector<int> N, B; vvd D;
    LPSolver(const &A, const &vd &b, const &vd &c) :
m(b.size()), n(c.size()), N(n+1), B(m), D(m+2, vd(n+2)){
        rep(i, m) rep(j, n) D[i][j] = A[i][j];
        rep(i, m) {
            B[i] = n + i; D[i][n] = -1; D[i][n + 1] = b[i];
        }
        rep(j, n) { N[j] = j; D[m][j] = -c[j]; }
        N[n] = -1; D[m + 1][n] = 1;
    }
    void pivot(int r, int s) {
        T *a = D[r].data(), inv = 1 / a[s];
        rep(i, m + 2) if (i != r && abs(D[i][s]) > eps) {
            T *b = D[i].data(), inv2 = b[s] * inv;
            repx(j, 0, n + 2) b[j] -= a[j] * inv2;
            b[s] = a[s] * inv2;
        }
        rep(j, n + 2) if (j != s) D[r][j] *= inv;
        rep(i, m + 2) if (i != r) D[i][s] *= -inv;
        D[r][s] = inv;
        swap(B[r], N[s]);
    }
    bool simplex(int phase) {
        int x = m + phase - 1;
        for (;;) {
            int s = -1;
            rep(j, n + 1) if (N[j] != -phase) ltj(D[x]);
            if (D[x][s] >= -eps) return true;
            int r = -1;
            rep(i, m) {
                if (D[i][s] <= eps) continue;
                if (r == -1 || MP(D[i][n + 1] / D[i][s],
B[i]) < MP(D[r][n + 1] / D[r][s], B[r])) r = i;
            }
            if (r == -1) return false;
            pivot(r, s);
        }
    }
    T solve(vd &x) {
        int r = 0;
        repx(i, 1, m) if (D[i][n + 1] < D[r][n + 1]) r = i;
        if (D[r][n + 1] < -eps) {
            pivot(r, n);
            if (!simplex(2) || D[m + 1][n + 1] < -eps)
return -inf;

```

```

        rep(i, m) if (B[i] == -1) {
            int s = 0;
            repx(j, 1, n + 1) ltj(D[i]);
            pivot(i, s);
        }
    }
    bool ok = simplex(1);
    x = vd(n);
    rep(i, m) if (B[i] < n) x[B[i]] = D[i][n + 1];
    return ok ? D[m][n + 1] : inf;
};

```

2.12 combinatorics

```

const int mxN = 1e6+5, M = 1e9+7;
ll f1[mxN], f2[mxN], iv[mxN];

ll binpow(ll a, ll b){
    ll res = 1;
    while(b){
        if(b & 1) res = res*a%M;
        a = a*a%M;
        b >>= 1;
    }
    return res;
}

ll modinv(ll x){
    return binpow(x, M-2);
}

ll nCr(int a, int b){
    return f1[a]*f2[b]%M*f2[a-b]%M;
    // // si b muy chico y probablemente a muy grande
    // if(b > a) return 0;
    // ll res = 1;
    // for(ll i = 0; i<b; ++i){
    //     ans *= (n-i)%M;
    //     ans %= M; ans += M; ans %= M;
    // }
    // for(ll i = 0; i<b; ++i){
    //     ans *= (modinv(i+1)) % M;
    //     ans %= M; ans += M; ans %= M;
    // }
    // return ans;
}

void solve(){
    int a, b;
    cin >> a >> b;
    cout << nCr(a, b) << "\n";
}

```

```

signed main(){
    iv[1] = 1;
    for(int i = 2; i<mxN; ++i){
        iv[i]=M-M/i*iv[M/i]%M;
    }
    f1[0]=f2[0]=1;
    for(int i = 1; i<mxN; ++i){
        f1[i]=f1[i-1]*i%M;
        f2[i]=f2[i-1]*iv[i]%M;
    }
    solve();
}

```

2.13 fht

```

ll c1[MAXN+9], c2[MAXN+9]; //MAXN must be power of 2!
void fht(ll* p, int n, bool inv){
    for(int l=1; 2*l<=n; l*=2) for(int
i=0; i<n; i+=2*l) fore(j, 0, l){
        ll u=p[i+j], v=p[i+l+j];
        if(!inv) p[i+j]=u+v, p[i+l+j]=u-v; // XOR
        else p[i+j]=(u+v)/2, p[i+l+j]=(u-v)/2;
        //if(!inv) p[i+j]=v, p[i+l+j]=u+v; // AND
        //else p[i+j]=-u+v, p[i+l+j]=u;
        //if(!inv) p[i+j]=u+v, p[i+l+j]=u; // OR
        //else p[i+j]=v, p[i+l+j]=u-v;
    }
}
// like polynomial multiplication, but XORing exponents
// instead of adding them (also ANDing, ORing)
vector<ll> multiply(vector<ll>& p1, vector<ll>& p2){
    int n=1<<(32-__builtin_clz(max(SZ(p1), SZ(p2))-1));
    fore(i, 0, n) c1[i]=0, c2[i]=0;
    fore(i, 0, SZ(p1)) c1[i]=p1[i];
    fore(i, 0, SZ(p2)) c2[i]=p2[i];
    fht(c1, n, false); fht(c2, n, false);
    fore(i, 0, n) c1[i]*=c2[i];
    fht(c1, n, true);
    return vector<ll>(c1, c1+n);
}

```

2.14 sieve

```

struct Sieve {
    vector<int> primes, smallestFactor;

    Sieve(int n) {
        smallestFactor.resize(n + 1);

        for (ll i = 2; i <= n; i++) {
            if (smallestFactor[i] == 0) {
                smallestFactor[i] = i;
                primes.push_back(i);
            }
        }
    }
}

```

```

    for (ll prime : primes) {
        if (prime * i > n || prime > smallestFactor[i])
            break;

        smallestFactor[prime * i] = prime;
    }
}

bool getIsPrime(int n) { return smallestFactor[n] == n; }

map<int, int> factorize(int n) {
    map<int, int> factors;

    while (n > 1) {
        int factor = smallestFactor[n];

        factors[factor]++;
        n /= factor;
    }

    return factors;
}
};

```

2.15 gauss mod 2

```

vector<vector<int>>
solveHomogeneousSystemMod2(vector<vector<int>> &A,
                             int
maxSolutions = 1e5) {
    int rows = A.size(), columns = A[0].size();

    int variables = columns - 1;

    vector<int> pivotPosition(rows, -1);

    int pivotRow = 0;

    for (int pivotColumn = 0; pivotColumn < variables &&
pivotRow < rows;
        pivotColumn++) {
        int pivotIndex = pivotRow;

        while (pivotIndex < rows && A[pivotIndex][pivotColumn]
== 0)
            pivotIndex++;

        if (pivotIndex == rows)
            continue;

        swap(A[pivotRow], A[pivotIndex]);

        pivotPosition[pivotRow] = pivotColumn;
    }
}

```

```

    for (int i = 0; i < rows; i++)
        if (i != pivotRow && A[i][pivotColumn] & 1)
            for (int j = pivotColumn; j < columns; j++)
                A[i][j] = A[i][j] ^ A[pivotRow][j];

    pivotRow++;
}

vector<bool> isPivot(variables);

for (int position : pivotPosition) {
    if (position == -1)
        continue;

    isPivot[position] = true;
}

vector<int> freeColumns;

for (int j = 0; j < variables; j++)
    if (!isPivot[j])
        freeColumns.push_back(j);

if (freeColumns.empty())

    return {vector<int>(variables)};

vector<vector<int>> solutions;

for (int freeColumn : freeColumns) {
    vector<int> solution(variables);

    solution[freeColumn] = 1;

    for (int i = 0; i < rows; i++) {
        if (pivotPosition[i] == -1)
            continue;

        int positionColumn = pivotPosition[i];
        solution[positionColumn] = A[i][freeColumn];
    }

    solutions.push_back(solution);

    if (solutions.size() == maxSolutions)
        break;
}

return solutions;
}

```

2.16 gauss

```

const double EPS = 1e-9;
// solve a system of equations.

```

```

// complexity: O(min(N, M) * N * M)
// `a` is a list of rows
// the last value in each row is the result of the equation
// return values:
// 0 -> no solutions
// 1 -> unique solution, stored in `ans`
// -1 -> infinitely many solutions, one of which is stored
in `ans`
// UNTESTED
int gauss(vector<vector<double>> a, vector<double> &ans) {
    int N = a.size(), M = a[0].size() - 1;
    vector<int> where(M, -1);
    for (int j = 0, i = 0; j < M && i < N; j++) {
        int sel = i;
        repx(k, i, N) if (abs(a[k][j]) > abs(a[sel][j])) sel
= k;

        if (abs(a[sel][j]) < EPS) continue;
        repx(k, j, M + 1) swap(a[sel][k], a[i][k]);
        where[j] = i;
        rep(k, N) if (k != i) {
            double c = a[k][j] / a[i][j];
            repx(l, j, M + 1) a[k][l] -= a[i][l] * c;
        }
        i++;
    }
    ans.assign(M, 0);
    rep(i, M) if (where[i] != -1) ans[i] = a[where[i]][M] /
a[where[i]][i];
    rep(i, N) {
        double sum = 0;
        rep(j, M) sum += ans[j] * a[i][j];
        if (abs(sum - a[i][M]) > EPS) return 0;
    }
    rep(i, M) if (where[i] == -1) return -1;
    return 1;
}

```

3 Geometry

3.1 convexhull

```

vector<Point> convexHull(vector<Point> points) {
    if (points.size() < 3)
        return points;

    sort(points.begin(), points.end());

    vector<Point> hull;

    // Lower hull
    for (Point &point : points) {
        // change <= to < for collinear
        while (hull.size() >= 2 &&
            point.left(hull[hull.size() - 2], hull.back()) <

```

```

0)
    hull.pop_back();

    hull.push_back(point);
}

hull.pop_back();

int m = hull.size();

// Upper hull
for (int i = points.size() - 1; i >= 0; --i) {
    Point &point = points[i];

    // change <= to < for collinear
    while (hull.size() >= m + 2 &&
        point.Left(hull[hull.size() - 2], hull.back()) <
0)
        hull.pop_back();

    hull.push_back(point);
}

hull.pop_back();

return hull;
}

```

3.2 minkowski

```

void reorder_polygon(vector<P> &ps) {
    int pos = 0;
    repx(i, 1, (int)ps.size()) {
        if (ps[i].y < ps[pos].y || (ps[i].y == ps[pos].y &&
ps[i].x < ps[pos].x))
            pos = i;
    }
    rotate(ps.begin(), ps.begin() + pos, ps.end());
}

vector<P> minkowski(vector<P> ps, vector<P> qs) {
    // the first vertex must be the lowest
    reorder_polygon(ps); reorder_polygon(qs);
    ps.push_back(ps[0]); ps.push_back(ps[1]);
    qs.push_back(qs[0]); qs.push_back(qs[1]);
    vector<P> result; int i = 0, j = 0;
    while (i < ps.size() - 2 || j < qs.size() - 2) {
        result.push_back(ps[i] + qs[j]);
        auto z = (ps[i + 1] - ps[i]) % (qs[j + 1] - qs[j]);
        if (z >= 0 && i < ps.size() - 2) ++i;
        if (z <= 0 && j < qs.size() - 2) ++j;
    }
    return result;
}

```

3.3 circle line intersection

```

template <class T>
optional<pair<Point<T>, Point<T>>>
circleLineIntersection(T radius, Point<T> first, Point<T>
second) {
    auto [x1, y1] = first;
    auto [x2, y2] = second;

    T dx = x2 - x1, dy = y2 - y1;

    T a = dy;
    T b = -dx;
    T c = -(a * x1 + b * y1);

    if (c * c > radius * radius * (a * a + b * b)) {
        return nullopt;
    }

    if (c * c == radius * radius * (a * a + b * b)) {
        // TODO: Check if this is correct
        T x0 = -a * c / (a * a + b * b), y0 = -b * c / (a * a +
b * b);
        return make_pair(Point<T>(x0, y0), Point<T>(x0, y0));
    }

    T d = radius * radius - c * c / (a * a + b * b);
    T mult = sqrt(d / (a * a + b * b));

    T x0 = -a * c / (a * a + b * b), y0 = -b * c / (a * a + b
* b);
    T ax = x0 + b * mult, ay = y0 - a * mult;
    T bx = x0 - b * mult, by = y0 + a * mult;

    return make_pair(Point<T>(ax, ay), Point<T>(bx, by));
}

```

3.4 circlehierarchy

```

int sweep_line_x = -1e8;
const double EPS = 1e-9;

struct Circle { int x, y, r;
    double get_y(bool up_side) const {
        int b = -2 * y;
        ll c = (ll) y * y + (sweep_line_x - x) *
ll(sweep_line_x - x) - (ll) r * r;
        ll delta = b * (ll) b - 4 * c;
        return (-b + (up_side ? sqrt(delta) : -
sqrt(delta))) / 2;
    }
};

struct Curve {
    bool up_side;
    Circle circle;

```

```

    bool operator<(const Curve &rhs) const {
        return circle.get_y(up_side) + EPS <
rhs.circle.get_y(rhs.up_side);
    }
};

struct Interval {
    int owner_id;
    Curve up, down;
    bool operator<(const Interval &rhs) const {
        return down < rhs.down || up < rhs.up;
    }
    bool operator==(const Interval &rhs) const {
        return !(*this < rhs) && !(rhs < *this);
    }
};

int n;
Circle circle[maxn];
vector<int> g[maxn];

int main() {
    cin >> n;
    vector<pair<int, int>> events;
    for (int i = 1; i <= n; i++) {
        cin >> circle[i].x >> circle[i].y >> circle[i].r;
        events.emplace_back(circle[i].x - circle[i].r, i);
        events.emplace_back(circle[i].x + circle[i].r, i);
    }
    sort(events.begin(), events.end());
    circle[0] = {0, 0, inf};
    set<Interval> intervals({{0, {true, circle[0]}, {false,
circle[0]}}});
    for (auto tmp : events) {
        int sw_x = tmp.first, circle_id = tmp.second;
        sweep_line_x = sw_x;
        int x = circle[circle_id].x, r =
circle[circle_id].r;
        Interval inside = {circle_id, {true,
circle[circle_id]}, {false, circle[circle_id]}};
        if (sw_x == x - r) {
            auto par_it = --intervals.lower_bound(inside);
            auto par_interval = *par_it;
            intervals.erase(par_it);
            g[par_interval.owner_id].push_back(circle_id);
            auto up_interval = par_interval, down_interval =
par_interval;
            up_interval.down = {true, circle[circle_id]};
            down_interval.up = {false, circle[circle_id]};
            intervals.insert(up_interval);
            intervals.insert(down_interval);
            intervals.insert(inside);
        } else {
            auto me = intervals.lower_bound(inside);
            auto joined = *prev(me);
            joined.up = next(me)->up;
            intervals.erase(prev(me));

```

```

        intervals.erase(next(me));
        intervals.erase(me);
        intervals.insert(joined);
    }
}
for (int i = 0; i <= n; ++i)
    for (auto child : g[i])
        cout << i << ' ' << child << '\n';
}

```

3.5 closest points

```

// sort by x
ll closest(vector<ii> &p) {
    int n = SZ(p);
    set<ii> s;
    ll best = 1e18;
    int j = 0;
    for(i, 0, n) {
        ll d = ceil(sqrt(best));
        while(p[i].fst - p[j].fst >= best)
            s.erase({p[j].snd, p[j].fst}), j++;
        auto it1=s.lower_bound({p[i].snd-d,p[i].fst});
        auto it2=s.upper_bound({p[i].snd+d,p[i].fst});
        for(auto it = it1; it != it2; ++it) {
            ll dx = p[i].fst - it->snd;
            ll dy = p[i].snd - it->fst;
            best = min(best, dx * dx + dy * dy);
        }
        s.insert({p[i].snd, p[i].fst});
    }
    return best;
}

```

3.6 point in polygon

```

// 0: outside, 1: inside, 2: boundary
template <class T>
int pointInPolygon(vector<Point<T>> &polygon, Point<T>
point) {
    int count = 0;
    bool isInBoundary = false;

    int n = polygon.size();

    for (int i = 0; i < n; i++) {
        Point<T> pointA = polygon[i], pointB = polygon[(i + 1) %
n];

        T cross = (pointB - pointA).cross(point - pointA);

        if (cross == 0 && point.x >= min(pointA.x, pointB.x) &&
            point.x <= max(pointA.x, pointB.x) &&
            point.y >= min(pointA.y, pointB.y) &&
            point.y <= max(pointA.y, pointB.y))

```

```

        return 2;

        if ((point.y <= pointA.y) == (point.y <= pointB.y))
            continue;

        T left = point.x * (pointB.y - pointA.y);

        T right = pointA.x * (pointB.y - pointA.y) +
            (point.y - pointA.y) * (pointB.x - pointA.x);

        if ((pointB.y - pointA.y) < 0)
            left *= -1, right *= -1;

        count += left < right;
    }

    if (isInBoundary)
        return 2;

    if (count % 2 == 0)
        return 0;

    return 1;
}

```

3.7 segment

```

int sign(ll x) {
    if (x == 0)
        return 0;

    return x > 0 ? 1 : -1;
}

bool intersect(ll a, ll b, ll c, ll d) {
    if (a > b)
        swap(a, b);

    if (c > d)
        swap(c, d);

    return max(a, c) <= min(b, d);
}

template <class T> struct Segment {
    Point<T> a, b;

    Segment() {}
    Segment(Point<T> a, Point<T> b) : a(a), b(b) {}

    friend istream &operator>>(istream &is, Segment &segment)
    {
        is >> segment.a >> segment.b;
        return is;
    }
}

```

```

bool intersects(const Segment &segment) const {
    Point<T> c = segment.a, d = segment.b;

    if (c.cross(a, d) == 0 && c.cross(b, d) == 0)
        return intersect(a.x, b.x, c.x, d.x) && intersect(a.y,
b.y, c.y, d.y);

    return sign(a.cross(b, c)) != sign(a.cross(b, d)) &&
        sign(c.cross(d, a)) != sign(c.cross(d, b));
}
};

```

3.8 halfplane intersection

```

// obtain the convex polygon that results from intersecting
the given list
// of halfplanes, represented as lines that allow their left
side
// assumes the halfplane intersection is bounded
vector<P> halfplane_intersect(vector<L> &H) {
    L bb(P(-INF, -INF), P(INF, 0));
    rep(k, 4) H.push_back(bb), bb.o = bb.o.rot(), bb.d =
bb.d.rot();

    sort(begin(H), end(H), [](L a, L b) { return
a.d.angcmp(b.d) < 0; });
    deque<L> q; int n = 0;
    rep(i, H.size()) {
        while (n >= 2 && H[i].side(q[n - 1].intersection(q[n
- 2])) > 0)
            q.pop_back(), n--;
        while (n >= 2 && H[i].side(q[0].intersection(q[1]))
> 0)
            q.pop_front(), n--;
        if (n > 0 && H[i].parallel(q[n - 1])) {
            if (H[i].d * q[n - 1].d < 0) return {};
            if (H[i].side(q[n - 1].o) > 0) q.pop_back(),
n--;
            else continue;
        }
        q.push_back(H[i]), n++;
    }

    while (n >= 3 && q[0].side(q[n - 1].intersection(q[n -
2])) > 0)
        q.pop_back(), n--;
    while (n >= 3 && q[n - 1].side(q[0].intersection(q[1]))
> 0)
        q.pop_front(), n--;
    if (n < 3) return {};

    vector<P> ps(n);
    rep(i, n) ps[i] = q[i].intersection(q[(i + 1) % n]);
}

```

```

    return ps;
}

```

3.9 point

```

template <class T> struct Point {
    T x, y;

    Point() {}
    Point(T x, T y) : x(x), y(y) {}

    friend istream &operator>>(istream &is, Point &point) {
        is >> point.x >> point.y;
        return is;
    }

    bool operator==(const Point &point) const {
        return point.x == x && point.y == y;
    }

    bool operator!=(const Point &point) const { return !(point
== *this); }

    bool operator<(const Point &point) const {
        if (x == point.x)
            return y < point.y;

        return x < point.x;
    }

    int left(const Point &point) const {
        T crossProduct = this->cross(point);

        if (crossProduct > 0)
            return 1;
        if (crossProduct < 0)
            return -1;

        return 0;
    }

    bool operator>(const Point &point) const {
        return !(*this > point) && (*this != point);
    }

    bool operator>=(const Point &point) {
        return *this > point || *this == point;
    }

    bool operator<=(const Point &point) {
        return *this < point || *this == point;
    }

    Point operator+(const Point &point) const {
        return Point(x + point.x, y + point.y);
}

```

```

}

Point operator-(const Point &point) const {
    return Point(x - point.x, y - point.y);
}

Point operator+(T value) const { return Point(x + value, y
+ value); }

Point operator-(T value) const { return Point(x - value, y
- value); }

T cross(const Point &point) const { return x * point.y - y
* point.x; }

T cross(const Point &a, const Point &b) const {
    return (a - *this).cross(b - *this);
}

T squaredLength() const { return x * x + y * y; }

T squaredDistance(const Point &point) {
    T deltaX = point.x - x, deltaY = point.y - y;

    return deltaX * deltaX + deltaY * deltaY;
}

T distance(const Point &point) { return
sqrt(squaredDistance(point)); }

void rotate(T radians) {
    T newX = x * cos(radians) - y * sin(radians),
    newY = x * sin(radians) + y * cos(radians);

    x = newX, y = newY;
};
}

```

3.10 polygon

```

template <class T> struct Polygon {
    vector<Point<T>> points, relative;
    int n;

    int sign(T value) {
        if (value == 0)
            return 0;

        return value > 0 ? 1 : -1;
    }

    Polygon(const int n) {
        this->n = n;
        points.resize(n);
    }
}

```

```

Polygon(const vector<Point<T>> &points) : points(points),
n(points.size()) {}

friend istream &operator>>(istream &is, Polygon &polygon)
{
    for (auto &point : polygon.points)
        is >> point;
    return is;
}

T doubleArea() {
    T sum = 0;

    for (int i = 0; i < n; i++)
        sum += points[i].cross(points[(i + 1) % n]);

    return abs(sum);
}

T area() { return doubleArea() / 2; }

T boundaryLatticePoints() {
    T count = 0;

    for (int i = 0; i < n; i++) {
        Point<T> delta = points[i] - points[(i + 1) % n];
        count += gcd(abs(delta.x), abs(delta.y));
    }

    return count;
}

T insideLatticePoints() {
    return (doubleArea() - boundaryLatticePoints()) / 2 + 1;
}

T latticePoints() { return boundaryLatticePoints() +
insideLatticePoints(); }
};

```

3.11 delaunay

```

// Returns planar graph representing Delaunay's
triangulation.
// Edges for each vertex are in ccw order.
// It can work with doubles, but also integers (replace long
double in line 51)

typedef struct QuadEdge* Q;
struct QuadEdge {
    int id,used;
    pt o;
    Q rot,nxt;
    QuadEdge(int id=-1, pt
o=pt(INF,INF)):id(id),used(0),o(o),rot(0),nxt(0){}
}

```



```

Q rev(){return rot->rot;}
Q next(){return nxt;}
Q prev(){return rot->next()->rot;}
pt dest(){return rev()->o;}
};

Q edge(pt a, pt b, int ida, int idb){
  Q e1=new QuadEdge(ida,a);
  Q e2=new QuadEdge(idb,b);
  Q e3=new QuadEdge;
  Q e4=new QuadEdge;
  tie(e1->rot,e2->rot,e3->rot,e4->rot)={e3,e4,e2,e1};
  tie(e1->nxt,e2->nxt,e3->nxt,e4->nxt)={e1,e2,e4,e3};
  return e1;
}

void splice(Q a, Q b){
  swap(a->nxt->rot->nxt,b->nxt->rot->nxt);
  swap(a->nxt,b->nxt);
}

void del_edge(Q& e, Q ne){
  splice(e,e->prev()); splice(e->rev(),e->rev()->prev());
  delete e->rev()->rot; delete e->rev();
  delete e->rot; delete e;
  e=ne;
}

Q conn(Q a, Q b){
  Q e=edge(a->dest(),b->o,a->rev()->id,b->id);
  splice(e,a->rev()->prev());
  splice(e->rev(),b);
  return e;
}

auto area(pt p, pt q, pt r){return (q-p)%(r-q);}

// is p in circumference formed by (a,b,c)?
bool in_c(pt a, pt b, pt c, pt p){
  // Warning: this number is 0(max_coord^4).
  // Consider using int128 or using an alternative method
  for this function
  long double p2=p*p,A=a*a-p2,B=b*b-p2,C=c*c-p2;
  return area(p,a,b)*C+area(p,b,c)*A+area(p,c,a)*B>EPS;
}

pair<Q,Q> build_tr(vector<pt>& p, int l, int r){
  if(r-l+1<=3){
    Q a=edge(p[l],p[l+1],l,l+1),b=edge(p[l+1],p[r],l+1,r);
    if(r-l+1==2) return {a,a->rev()};
    splice(a->rev(),b);
    auto ar=area(p[l],p[l+1],p[r]);
    Q c=abs(ar)>EPS?conn(b,a):0;
    if(ar>=-EPS) return {a,b->rev()};
    return {c->rev(),c};
  }

```

```

}
int m=(l+r)/2;
auto [la,ra]=build_tr(p,l,m);
auto [lb,rb]=build_tr(p,m+1,r);
while(1){
  if(ra->dest().left(lb->o,ra->o)) ra=ra->rev()->prev();
  else if(lb->dest().left(lb->o,ra->o)) lb=lb->rev()->next();
  else break;
}
Q b=conn(lb->rev(),ra);
auto valid=[&](Q e){return b->o.left(e->dest(),b->dest());};
if(ra->o==la->o) la=b->rev();
if(lb->o==rb->o) rb=b;
while(1){
  Q L=b->rev()->next();
  if(valid(L)) while(in_c(b->dest(),b->o,L->dest(),L->next()->dest())) del_edge(L,L->next());
  Q R=b->prev();
  if(valid(R)) while(in_c(b->dest(),b->o,R->dest(),R->prev()->dest())) del_edge(R,R->prev());
  if(!valid(L)&&!valid(R)) break;
  if(!valid(L)|| (valid(R)&&in_c(L->dest(),L->o,R->o,R->dest()))) b=conn(R,b->rev());
  else b=conn(b->rev(),L->rev());
}
return {la,rb};
}

vector<vector<int>> delaunay(vector<pt> v){
  int n=SZ(v); auto tmp=v;
  vector<int> id(n); iota(ALL(id),0);
  sort(ALL(id),[&](int l, int r){return v[l]<v[r]});
  for(i,0,n) v[i]=tmp[id[i]];
  assert(unique(ALL(v))==v.end());
  vector<vector<int>> g(n);
  int col=1;
  for(i,2,n) col&=abs(area(v[i],v[i-1],v[i-2]))<=EPS;
  if(col){
    for(i,1,n) g[id[i-1]].pb(id[i]),g[id[i]].pb(id[i-1]);
  }
  else{
    Q e=build_tr(v,0,n-1).fst;
    vector<Q> edg={e};
    for(int i=0;i<SZ(edg);e=edg[i++){
      for(Q at=e;!at->used;at=at->next()){
        at->used=1;
        g[id[at->id]].pb(id[at->rev()->id]);
        edg.pb(at->rev());
      }
    }
  }
  return g;
}

```

4 Algorithms

4.1 ternary search

```

template <typename T = double>
typename enable_if<is_floating_point<T>::value>
ternarySearch(function<T(T)> calculate, T left, T right,
bool searchMin = true,
    T epsilon = 1e-9) { // [left, right]
  while (right - left > epsilon) {
    T midA = left + (right - left) / 3, midB = right -
(right - left) / 3;

    T valueA = calculate(midA), valueB = calculate(midB);

    if (searchMin)
      valueA *= -1, valueB *= -1;

    if (valueA < valueB)
      left = midA;
    else
      right = midB;
  }

  return calculate(left);
}

template <typename T = int>
typename enable_if<is_integral<T>::value>
ternarySearch(function<T(T)> calculate, T left, T right,
bool searchMin = true,
    T epsilon = 5) { // [left, right]
  while (right - left > epsilon) {
    T midA = left + (right - left) / 3, midB = right -
(right - left) / 3;

    T valueA = calculate(midA), valueB = calculate(midB);

    if (searchMin)
      valueA *= -1, valueB *= -1;

    if (valueA < valueB)
      left = midA;
    else
      right = midB;
  }

  T answer = calculate(left);

  for (T i = left + 1; i <= right; i++) {
    T value = calculate(i);

    if (searchMin)
      answer = min(answer, value);
  }
}

```



```

else
    answer = max(answer, value);
}

return answer;
}

```

5 Graphs

5.1 mincostmaxflow

```

template <class T>
class MCMF{
    typedef pair<T, T> pTT;
    T INF = numeric_limits<T>::max();
    struct Edge{
        int v; T c, w;
        Edge(int v, T c, T w) : v(v), c(c), w(w) {}
    };
    int n; vvi E;
    vector<Edge> L; vi F; vector<T> D, P; vector<bool> V;
    bool dij(int s, int t){
        D.assign(n, INF); F.assign(n, -1); V.assign(n,
false);
        // D[s] = 0;
        // rep(_, n){
        //     int best = -1;
        //     rep(i, n) if (!V[i] && (best == -1 || D[best]
> D[i])) best = i;
        //     if (D[best] >= INF) break;
        //     V[best] = true;
        //     for (int e : E[best]){
        //         Edge ed = L[e];
        //         if (ed.c == 0) continue;
        //         T toD = D[best] + ed.w + P[best] -
P[ed.v];
        //         if (toD < D[ed.v]) D[ed.v] = toD, F[ed.v]
= e;
        //     }
        // }
        priority_queue<pair<T, int>> q;
        D[s] = 0;
        q.push({D[s], s});
        while(!q.empty()) {
            int x = q.top().second;
            ll d = -q.top().first;
            q.pop();
            if(D[x] != d) continue;
            for(int e: E[x]) {
                Edge ed = L[e];
                if(ed.c == 0) continue;
                T toD = D[x] + ed.w + P[x] - P[ed.v];
                if(D[ed.v] > toD) {
                    D[ed.v] = toD;

```

```

                q.push({-D[ed.v], ed.v});
                F[ed.v] = e;
            }
        }
        return D[t] < INF;
    }
    pTT augment(int s, int t){
        pTT flow(L[F[t]].c, 0);
        for (int v = t; v != s; v = L[F[v] ^ 1].v)
            flow.ff = min(flow.ff, L[F[v]].c), flow.ss +=
L[F[v]].w;
        for (int v = t; v != s; v = L[F[v] ^ 1].v)
            L[F[v]].c -= flow.ff, L[F[v] ^ 1].c += flow.ff;
        return flow;
    }
public:
    MCMF(int n) : n(n), E(n), D(n), P(n, 0), V(n, 0) {}
    pTT mcmf(int s, int t){
        pTT ans(0, 0);
        if (!dij(s, t)) return ans;
        rep(i, n) if (D[i] < INF) P[i] += D[i];
        while (dij(s, t)){
            auto flow = augment(s, t);
            ans.ff += flow.ff, ans.ss += flow.ff * flow.ss;
            rep(i, n) if (D[i] < INF) P[i] += D[i];
        }
        return ans;
    }
    void addEdge(int u, int v, T c, T w){
        E[u].pb(L.size()); L.pb(v, c, w);
        E[v].pb(L.size()); L.pb(u, 0, -w);
    }
};

```

5.2 chuliu

```

//0(n*m) minimum spanning tree in directed graph
//returns -1 if not possible
//included i-th edge if take[i]!=0
typedef int tw; tw INF=1ll<<30;
struct edge{int u,v,id;tw len;};
struct ChuLiu{
    int n; vector<edge> e;
    vector<int> inc,dec,take,pre,num,id,vis;
    vector<tw> inw;
    void add_edge(int x, int y, tw w){
        inc.pb(0); dec.pb(0); take.pb(0);
        e.pb({x,y,SZ(e),w});
    }
    ChuLiu(int n):n(n),pre(n),num(n),id(n),vis(n),inw(n){}
    tw doit(int root){
        auto e2=e;
        tw ans=0; int eg=SZ(e)-1,pos=SZ(e)-1;
        while(1){

```

```

            fore(i,0,n) inw[i]=INF,id[i]=vis[i]=-1;
            for(auto ed:e2) if(ed.len<inw[ed.v]){
                inw[ed.v]=ed.len; pre[ed.v]=ed.u;
                num[ed.v]=ed.id;
            }
            inw[root]=0;
            fore(i,0,n) if(inw[i]==INF) return -1;
            int tot=-1;
            fore(i,0,n){
                ans+=inw[i];
                if(i!=root)take[num[i]]++;
                int j=i;
                while(vis[j]!=i&&j!
=root&&id[j]<0)vis[j]=i,j=pre[j];
                if(j!=root&&id[j]<0){
                    id[j]=++tot;
                    for(int k=pre[j];k!=j;k=pre[k])
id[k]=tot;
                }
            }
            if(tot<0)break;
            fore(i,0,n) if(id[i]<0)id[i]=++tot;
            n=tot+1; int j=0;
            fore(i,0,SZ(e2)){
                int v=e2[i].v;
                e2[j].v=id[e2[i].v];
                e2[j].u=id[e2[i].u];
                if(e2[j].v!=e2[j].u){
                    e2[j].len=e2[i].len-inw[v];
                    inc.pb(e2[i].id);
                    dec.pb(num[v]);
                    take.pb(0);
                    e2[j++].id=++pos;
                }
            }
            e2.resize(j);
            root=id[root];
        }
        while(pos>eg){
            if(take[pos]>0) take[inc[pos]]++;
            take[dec[pos]]--;
            pos--;
        }
        return ans;
    }
};

```

5.3 scc

```

const int mxN = 2e5+5;

int n, m, who[mxN], cc = 0;
vector<int> adj[mxN], adj_rev[mxN], adj2[mxN];
bool used[mxN];
vector<int> order, component, root_nodes;

```

```

void dfs1(int v) {
    used[v] = true;
    for (auto u : adj[v])
        if (!used[u])
            dfs1(u);
    order.push_back(v);
}

void dfs2(int v) {
    used[v] = true;
    component.push_back(v);
    for (auto u : adj_rev[v]){
        if (!used[u]){
            dfs2(u);
        }
    }
}

int main(){
    std::ios_base::sync_with_stdio(false); std::cin.tie(0);
    std::cout.tie(0);

    cin >> n >> m;
    for(int i = 0; i<m; ++i){
        int a, b;
        cin >> a >> b, --a, --b;
        adj[a].push_back(b);
        adj_rev[b].push_back(a);
    }
    for(int i = 0; i<n; ++i){
        if(!used[i]){
            dfs1(i);
        }
    }
    memset(used, false, sizeof(used));
    reverse(order.begin(), order.end());
    for(int v : order){
        if(!used[v]){
            dfs2(v);
            int root = component.front();
            for(int u : component){
                who[u] = root;
            }
            root_nodes.push_back(root);
            component.clear();
            cc++;
        }
    }

    // condensation graph construction
    for(int v = 0; v<n; ++v){
        for(auto u : adj[v]){
            int root_v = who[v];
            int root_u = who[u];

```

```

        if (root_u != root_v){
            adj2[root_v].push_back(root_u);
        }
    }
}

```

5.4 dsu

```

struct DisjointUnionSet {
    int n;
    vector<int> parent, size;

    DisjointUnionSet(int n) : n(n) {
        parent.resize(n);
        size.resize(n);

        for (int i = 0; i < n; i++) {
            parent[i] = i;
            size[i] = 1;
        }
    }

    int Find(int u) {
        if (parent[u] == u)
            return u;

        return parent[u] = Find(parent[u]);
    }

    int Size(int u) { return size[Find(u)]; }
    void Union(int u, int v) {
        u = Find(u);
        v = Find(v);

        if (u == v)
            return;

        if (size[u] < size[v])
            swap(u, v);

        parent[v] = u;
        size[u] += size[v];
    }

    bool areConnected(int u, int v) { return Find(u) == Find(v); }
};

```

5.5 dominatortree

```

//idom[i]=parent of i in dominator tree with root=rt, or -1
//if not exists
int
n, rnk[MAXN], pre[MAXN], anc[MAXN], idom[MAXN], semi[MAXN], low[MAXN];
vector<int> g[MAXN], rev[MAXN], dom[MAXN], ord;
void dfspre(int pos){

```

```

    rnk[pos]=SZ(ord); ord.pb(pos);
    for(auto x:g[pos]){
        rev[x].pb(pos);
        if(rnk[x]==n) pre[x]=pos, dfspre(x);
    }
}

int eval(int v){
    if(anc[v]<n&&anc[anc[v]]<n){
        int x=eval(anc[v]);
        if(rnk[semi[low[v]]]>rnk[semi[x]]) low[v]=x;
        anc[v]=anc[anc[v]];
    }
    return low[v];
}

void dominators(int rt){
    fore(i,0,n){
        dom[i].clear(); rev[i].clear();
        rnk[i]=pre[i]=anc[i]=idom[i]=n;
        semi[i]=low[i]=i;
    }
    ord.clear(); dfspre(rt);
    for(int i=SZ(ord)-1; i--){
        int w=ord[i];
        for(int v:rev[w]){
            int u=eval(v);
            if(rnk[semi[w]]>rnk[semi[u]])semi[w]=semi[u];
        }
        dom[semi[w]].pb(w); anc[w]=pre[w];
        for(int v:dom[pre[w]]){
            int u=eval(v);
            idom[v]=(rnk[pre[w]]>rnk[semi[u]]?u:pre[w]);
        }
        dom[pre[w]].clear();
    }
    for(int w:ord) if(w!=rt&&idom[w]!=semi[w])
        idom[w]=idom[idom[w]];
    fore(i,0,n) if(idom[i]==n)idom[i]=-1;
}

```

5.6 bridges

```

void is_bridge(int v, int to); // some function to process
the found bridge
int n; // number of nodes
vector<vector<int>> adj; // adjacency list of graph

vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    bool parent_skipped = false;
    for (int to : adj[v]) {

```

```

    if (to == p && !parent_skipped) {
        parent_skipped = true;
        continue;
    }
    if (visited[to]) {
        low[v] = min(low[v], tin[to]);
    } else {
        dfs(to, v);
        low[v] = min(low[v], low[to]);
        if (low[to] > tin[v])
            is_bridge(v, to);
    }
}
}

```

```

void find_bridges() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            dfs(i);
    }
}

```

5.7 hungarian

```

// Minimum/Maximum cost of a perfect matching in complete
graph
// O(n^3)
template<class T>
class Hungarian{
    T inf = numeric_limits<T>::max() / 2;
    bool maxi, swapped = false;
    vector<vector<T>> cost;
    vector<T> u, v;
    vl p, way;
    int l, r;

public:
    // left/right == partition sizes
    Hungarian(int left, int right, bool maximizing){
        l = left, r = right, maxi = maximizing;
        if (swapped = l > r) swap(l, r);
        cost.assign(l + 1, vector<T>(r + 1, 0));
        u.assign(l + 1, 0); v.assign(r + 1, 0);
        p.assign(r + 1, 0); way.assign(r + 1, 0);
    }
    void add_edge(int l, int r, T w){
        assert(l and r); // indices start from 1 !!
        if (swapped) swap(l, r);
        cost[l][r] = maxi ? -w : w;
    }
    // execute after all edges were added

```

```

    void calculate(){
        repx(i, 1, l + 1){
            vector<bool> used(r + 1, false);
            vector<T> minv(r + 1, inf);
            int j0 = 0; p[0] = i;
            while (p[j0]){
                int j1, i0 = p[j0]; used[j0] = true;
                T delta = inf;
                repx(j, 1, r + 1) if (not used[j])
                {
                    T cur = cost[i0][j] - u[i0] - v[j];
                    if (cur < minv[j]) minv[j] = cur, way[j]
                    = j0;
                    if (minv[j] < delta) delta = minv[j], j1
                    = j;
                }
                rep(j, r + 1){
                    if (used[j]) u[p[j]] += delta, v[j] -=
                    delta;
                    else minv[j] -= delta;
                }
                j0 = j1;
                while (j0) p[j0] = p[way[j0]], j0 = way[j0];
            }
        }

        // execute after executing calculate()
        T answer() { return maxi ? v[0] : -v[0]; }
        bool are_matched(int l, int r){
            if (swapped) swap(l, r);
            return p[r] == l;
        }
    };

    int main(){
        ios_base::sync_with_stdio(0); cin.tie(0);
        ll n;
        cin >> n;
        ll d[n][n], pos = (n + 1) / 2;
        Hungarian<ll> h(pos, pos, 0);
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                cin >> d[i][j];
        for (int i = 0; i < n; i += 2){
            for (int j = 0; j < n; j += 2){
                ll cost = 0;
                if (j > 0) cost += d[i][j - 1];
                if (j < n - 1) cost += d[i][j + 1];
                h.add_edge(i / 2 + 1, j / 2 + 1, cost);
            }
        }
        h.calculate();
        cout << h.answer() << "\n";
    }
}

```

5.8 floydwarshall

```

// calculate distances between every pair of nodes in O(V^3)
time.
// works with negative edges, but not negative cycles.
void floyd(const vector<vector<pair<ll, int>>> &G,
vector<vector<ll>> &D) {
    int N = G.size();
    D.assign(N, vector<ll>(N, INF));
    rep(u, N) D[u][u] = 0;
    rep(u, N) for (auto [w, v] : G[u]) D[u][v] = w;
    rep(k, N) rep(u, N) rep(v, N)
        D[u][v] = min(D[u][v], D[u][k] + D[k][v]);
}

```

5.9 2sat

```

// assign boolean values to variables who are given in CNF (a
v b) ^ (not a v b)
struct TwoSatSolver {
    int n_vars;
    int n_vertices;
    vector<vector<int>> adj, adj_t;
    vector<bool> used;
    vector<int> order, comp;
    vector<bool> assignment;

    TwoSatSolver(int _n_vars) : n_vars(_n_vars),
n_vertices(2 * n_vars), adj(n_vertices), adj_t(n_vertices),
used(n_vertices), order(), comp(n_vertices, -1),
assignment(n_vars){
        order.reserve(n_vertices);
    }
    void dfs1(int v) {
        used[v] = true;
        for (int u : adj[v]) {
            if (!used[u])
                dfs1(u);
        }
        order.push_back(v);
    }
    void dfs2(int v, int cl) {
        comp[v] = cl;
        for (int u : adj_t[v]) {
            if (comp[u] == -1)
                dfs2(u, cl);
        }
    }
    bool solve_2SAT() {
        order.clear();
        used.assign(n_vertices, false);
        for (int i = 0; i < n_vertices; ++i) {
            if (!used[i])

```

```

        dfs1(i);
    }

    comp.assign(n_vertices, -1);
    for (int i = 0, j = 0; i < n_vertices; ++i) {
        int v = order[n_vertices - i - 1];
        if (comp[v] == -1)
            dfs2(v, j++);
    }

    assignment.assign(n_vars, false);
    for (int i = 0; i < n_vertices; i += 2) {
        if (comp[i] == comp[i + 1])
            return false;
        assignment[i / 2] = comp[i] > comp[i + 1];
    }
    return true;
}

void add_disjunction(int a, bool na, int b, bool nb) {
    // na and nb signify whether a and b are to be
    negated
    a = 2 * a ^ na;
    b = 2 * b ^ nb;
    int neg_a = a ^ 1;
    int neg_b = b ^ 1;
    adj[neg_a].push_back(b);
    adj[neg_b].push_back(a);
    adj_t[b].push_back(neg_a);
    adj_t[a].push_back(neg_b);
}

static void example_usage() {
    TwoSatSolver solver(3); // a, b, c
    solver.add_disjunction(0, false, 1, true); // a
v not b
    solver.add_disjunction(0, true, 1, true); // not a
v not b
    solver.add_disjunction(1, false, 2, false); // b
v c
    solver.add_disjunction(0, false, 0, false); // a
v a
    assert(solver.solve_2SAT() == true);
    auto expected = vector<bool>(True, False, True);
    assert(solver.assignment == expected);
}
};

```

5.10 partiallypersistentdsu

```

struct PPDSU {
    vector<vector<pair<int, int>>> par;
    int time = 0; //initial time
    PPDSU(int n) : par(n + 1, {{-1, 0}}) {}
    bool merge(int u, int v) {

```

```

        ++time;
        if ((u = root(u, time)) == (v = root(v, time))) return
0;
        if (par[u].back().first > par[v].back().first) swap(u,
v);
        par[u].push_back({par[u].back().first +
par[v].back().first, time});
        par[v].push_back({u, time}); //par[v] = u
        return 1;
    }
    bool same(int u, int v, int t) {
        return root(u, t) == root(v, t);
    }
    int root(int u, int t) { //root of u at time t
        if (par[u].back().first >= 0 && par[u].back().second <=
t) return root(par[u].back().first, t);
        return u;
    }
    int size(int u, int t) { //size of the component of u at
time t
        u = root(u, t);
        int l = 0, r = (int) par[u].size() - 1, ans = 0;
        while (l <= r) {
            int mid = l + r >> 1;
            if (par[u][mid].second <= t) ans = mid, l = mid + 1;
            else r = mid - 1;
        }
        return -par[u][ans].first;
    }
};

```

5.11 cut points

```

int n; // number of nodes
vector<vector<int>> adj; // adjacency list of graph

vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children = 0;
    for (int to : adj[v]) {
        if (to == p)
            continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p != -1)
                is_cutpoint(v); // change
            ++children;

```

```

    }
}
if (p == -1 && children > 1)
    is_cutpoint(v); // change
}

void find_cutpoints() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            dfs(i);
    }
}

```

5.12 dinic

```

// time: O(E V^2)
// O(E V^(2/3)) / O(E sqrt(E)) unit capacities
// O(E sqrt(V)) (hopcroft-karp) unit networks
// unit network: c in {0,1} & forall v, indeg<=1 or
outdeg<=1
// min-cut: nodes reachable from s in final residual graph

// en grafo bipartito:
// max matching = max flow
// min edge cover = conjunto minimo de arista que cubran
todos los nodos
// min edge cover = n - max_matching
// min vertex cover = conjunto minimo de nodos que cubran
todas las aristas
// min vertex cover = max_matching
// maximal independent set = conjunto de vertices que no
compartan aristas
// maximal independent set = min edge cover

// dilworth: pongo todos los nodos a la izquierda y una
copia a la derecha
// se une un nodo x de la izquierda a y de la derecha si
existe camino de x a y
// anticadena = conjunto de nodos tal que no hay camino
entre ellos
// cadena = conjunto de nodos tal que de cada uno puedo
llegar al siguiente
// anticadena de mayor tamaño = minima cantidad de
anticadenas que cubren todos los nodos
// min chain cover = conseguimos el max matching, si existe
x -> y, y -> z, en el matching
// consideramos x -> y -> z como una sola cadena (con
dsu o un vector posiblemente)
// Anticadena maxima = hacemos min vertex cover y tomamos
los nodos que no tienen ninguna
// copia en el cubrimiento, este es el

```

```

tamaño maximo

// Flujo con demandas (no necesariamente el maximo)
// Agregar s' y t' nuevos source and sink
// c'(s', v) = sum(d(u, v) for u in V) forall arista (s', v)
// c'(v, t') = sum(d(v, w) for w in V) forall arista (v, t')
// c'(u, v) = c(u, v) - d(u, v) forall aristas antiguas
// c'(t, s) = INF (el flujo por esta arista es el flujo real)*/
class Dinic{
    struct Edge { int to, rev; ll f, c; };
    int n, t_; vector<vector<Edge>> G;
    vl D; vl q, W;
    bool bfs(int s, int t){
        W.assign(n, 0); D.assign(n, -1); D[s] = 0;
        int f = 0, l = 0; q[l++] = s;
        while (f < l){
            int u = q[f++];
            for (const Edge &e : G[u]) if (D[e.to] == -1 &&
e.f < e.c)
                D[e.to] = D[u] + 1, q[l++] = e.to;
        }
        return D[t] != -1;
    }
    ll dfs(int u, ll f){
        if (u == t_) return f;
        for (int &i = W[u]; i < (int)G[u].size(); ++i){
            Edge &e = G[u][i]; int v = e.to;
            if (e.c <= e.f || D[v] != D[u] + 1) continue;
            ll df = dfs(v, min(f, e.c - e.f));
            if (df > 0) { e.f += df, G[v][e.rev].f -= df;
return df; }
        }
        return 0;
    }
public:
    Dinic(int N) : n(N), G(N), D(N), q(N) {}
    void addEdge(int u, int v, ll cap){
        G[u].push_back({v, (int)G[v].size(), 0, cap});
        G[v].push_back({u, (int)G[u].size() - 1, 0, 0}); //
cap if bidirectional
    }
    ll maxFlow(int s, int t){
        t_ = t; ll ans = 0;
        while (bfs(s, t)) while (ll dl = dfs(s, LLONG_MAX))
ans += dl;
        return ans;
    }
};

```

5.13 specialpathwithcentroids

```

int n, k, sz[mxN], mxtree[mxN], ans;
vector<int> adj[mxN];
bool used[mxN];

int dfs(int u, int p = -1){
    sz[u] = 1;
    for(int v : adj[u]){
        if(v != p && !used[v]){
            sz[u] += dfs(v, u);
        }
    }
    return sz[u];
}

int get_centroid(int u, int ms, int p = -1){ // u = node, ms
= size of the current tree delimited by previous centroids,
p = parent
    for(int v : adj[u]){
        if(v != p && !used[v]){
            if(sz[v]*2 > ms) return get_centroid(v, ms, u);
        }
    }
    return u;
}

void solve(int u){
    int sz_curr_tree = dfs(u);
    int centroid = get_centroid(u, sz_curr_tree);
    used[centroid] = 1;
    // do something with the centroid
    for(int v : adj[centroid]){
        // end that something with the centroid

    }
    // solve for child trees centroids
    for(int v : adj[centroid]){
        if(used[v]) continue;
        solve(v);
    }
}

```

5.14 heavylightdecomposition

```

template <class ST, class T, T merge(T a, T b)>
struct HeavyLightDecomposition
{
    vector<vector<int>> tree;
    vector<int> parent, depth, heavy, head, position, size;
    int currentPosition;
    ST segmentTree;

    HeavyLightDecomposition() {}
    HeavyLightDecomposition(vector<vector<int>> &tree) :
tree(tree)

```

```

{
    int n = tree.size();

    parent.resize(n);
    depth.resize(n);
    heavy.assign(n, -1);
    head.resize(n);
    position.resize(n);
    size.resize(n);
    currentPosition = 0;

    dfs(0);
    decompose(0, 0);
}

int dfs(int node)
{
    size[node] = 1;
    int maxSubtreeSize = 0;

    for (int child : tree[node])
    {
        if (child == parent[node])
            continue;

        parent[child] = node;
        depth[child] = depth[node] + 1;

        int subtreeSize = dfs(child);
        size[node] += subtreeSize;

        if (subtreeSize <= maxSubtreeSize)
            continue;

        maxSubtreeSize = subtreeSize;
        heavy[node] = child;
    }

    return size[node];
}

void decompose(int node, int nodeHead)
{
    head[node] = nodeHead;
    position[node] = currentPosition++;

    if (heavy[node] != -1)
        decompose(heavy[node], nodeHead);

    for (int child : tree[node])
        if (child != parent[node] && child != heavy[node])
            decompose(child, child);
}

int query(int a, int b)

```

```

{
    T answer;
    bool hasValue = false;

    while (head[a] != head[b])
    {
        if (depth[head[a]] > depth[head[b]])
            swap(a, b);

        T current = segmentTree.query(position[head[b]],
            position[b]);

        answer = hasValue ? merge(answer, current) : current;
        hasValue = true;

        b = parent[head[b]];
    }

    if (depth[a] > depth[b])
        swap(a, b);

    T last = segmentTree.query(position[a], position[b]);

    return hasValue ? merge(answer, last) : last;
}

vector<T> sorted(vector<T> &values)
{
    int n = values.size();

    vector<T> result(n);

    for (int i = 0; i < n; i++)
        result[position[i]] = values[i];

    return result;
}

void update(int node, T value)
{ segmentTree.update(position[node], value); }

// void update(int subtree, T value) {
//     segmentTree.update(position[subtree],
// position[subtree] + size[subtree] -
//     1,
//     value);
// }

// HeavyLightDecomposition<SegmentTree<int, merge>, int,
// merge> heavyLight(tree);
// heavyLight.segmentTree = SegmentTree<int,
// merge>(heavyLight.sorted(values));

```

6 Dp

6.1 knuth

```

int N; vector<int> A;
vector<vector<int>> DP, OPT;

int main(){
    DP.assign(N + 1, vi(N + 1));
    OPT.assign(N + 1, vi(N + 1));
    rep(i, N) DP[i][i + 1] = A[i + 1] - A[i], OPT[i][i + 1]
    = i;
    repx(d, 2, N + 1) rep(l, N + 1 - d){
        int r = l + d, l_ = OPT[l][r - 1], r_ = OPT[l + 1]
    [r];
        DP[l][r] = 1e9;
        repx(i, l_, r_ + 1){
            int aux = DP[l][i] + DP[i][r] + A[r] - A[l];
            if (aux < DP[l][r]) DP[l][r] = aux, OPT[l][r] =
            i;
        }
    }
}

```

6.2 digitdp

```

int dp[12][12][2]; // dp[i][s][f] {i: posicion, s: estado
del problema, f: act < s}
int k;
vector<int> num;

// solve(r) - solve(l-1)
int call(int pos, int cnt, int f){
    if(cnt > k) return 0;
    if(pos == num.size()){
        if(cnt == k) return 1;
        return 0;
    }
    if(dp[pos][cnt][f] != -1) return dp[pos][cnt][f];
    int res = 0, LMT;
    if(f == 0) LMT = num[pos];
    else LMT = 9;
    /// Try to place all the valid digits such that the
    number doesn't exceed b
    for(int dgt = 0; dgt <= LMT; dgt++){
        int nf = f, ncnt = cnt;
        if(f == 0 && dgt < LMT) nf = 1; /// The number is
        getting smaller at this position
        if(dgt == d) ncnt++;
        if(ncnt <= k) res += call(pos+1, ncnt, nf);
    }
    return dp[pos][cnt][f] = res;
}

int solve(string s){

```

```

num.clear();
while(s.size()){
    num.push_back((s.back() - '0') % 10);
    s.pop_back();
}
reverse(num.begin(), num.end());
memset(dp, -1, sizeof(dp));
return call(0, 0, 0);
}

```

6.3 linecontainer

```

struct Line {
    mutable ll a, b, c;

    bool operator<(Line r) const { return a < r.a; }
    bool operator<(ll x) const { return c < x; }
};

// dynamically insert `a*x + b` lines and query for maximum
// at any x all operations have complexity O(log N)
struct LineContainer : multiset<Line, less<>> {
    ll div(ll a, ll b) {
        return a / b - ((a ^ b) < 0 && a % b);
    }

    bool isect(iterator x, iterator y) {
        if (y == end()) return x->c = INF, 0;
        if (x->a == y->a) x->c = x->b > y->b ? INF : -INF;
        else x->c = div(y->b - x->b, x->a - y->a);
        return x->c >= y->c;
    }

    void add(ll a, ll b) {
        // a *= -1, b *= -1 // for min
        auto z = insert({a, b, 0}), y = z++, x = y;
        while (isect(y, z)) z = erase(z);
        if (x != begin() && isect(--x, y)) isect(x, y =
        erase(y));
        while ((y = x) != begin() && (--x)->c >= y->c)
        isect(x, erase(y));
    }

    ll query(ll x) {
        if (empty()) return -INF; // INF for min
        auto l = *lower_bound(x);
        return l.a * x + l.b;
        // return -l.a * x - l.b; // for min
    }
};

```

6.4 1d1d

```

#include <bits/stdc++.h>
using namespace std;

```



```

#define ll long long
#define ar array
#define rep(i, n) for(int i = 0; i<(int)n; ++i)
#define repx(i, a, b) for(int i = (int)a; i<(int)b; ++i)

const int mxN = 2e5+5, M = 1e9+7;
int n, s, dp[mxN], t[mxN], f[mxN];

int w(int j, int i){ //
    if(j >= i) return 1e9; // here is the cost function
    from (j, i)
    return s*(f[n]-f[j]) + t[i]*(f[i]-f[j]);
}

struct DD{ // index 0 is used as neutral state
    vector<pair<int, int>> v; // (start pos, best k)
    DD() { v.push_back(make_pair(0, 0)); }
    int qry(int i){
        return (--lower_bound(v.begin(), v.end(),
make_pair(i+1, 0)))>-second;
    }
    void upd(int x){
        for(int i = (int)v.size()-1; i>=0; --i){
            int y = v[i].first, oldk = v[i].second;
            if(y > x && dp[x] + w(x, y) < dp[oldk] + w(oldk,
y)) v.pop_back();
            else{
                int l = y+1, r = n+1;
                while(l < r){
                    int mid = (l+r)/2;
                    if(dp[x] + w(x, mid) < dp[oldk] +
w(oldk, mid)) r = mid;
                    else l = mid+1;
                }
                if(r != n+1) v.push_back(make_pair(r, x));
            }
        }
        if(v.size() == 0) v.push_back(make_pair(0, x));
    }
};

cin >> n >> s;
DD dd;
t[0] = f[0] = dp[0] = 0;
for(int i = 1; i<=n; ++i){
    cin >> t[i] >> f[i]; t[i] += t[i-1]; f[i] += f[i-1];
}
for(int i = 1; i<=n; ++i){
    int k = dd.qry(i); dp[i] = dp[k] + w(k,
i);dd.upd(i);
}
cout << dp[n] << '\n';
// https://vjudge.net/problem/OpenJ_Bailian-1180

```

6.5 broken profile

```

// Common problems solved using DP on broken profile
include:

// finding number of ways to fully fill an area (e.g.
chessboard/grid) with some figures (e.g. dominoes)
// finding a way to fill an area with minimum number of
figures
// finding a partial fill with minimum number of unfilled
space (or cells, in case of grid)
// finding a partial fill with the minimum number of
figures, such that no more figures can be added

int n, m;
vector<vector<ll>> dp;

void calc (int x = 0, int y = 0, int mask = 0, int next_mask
= 0){
    if (x == n) return;
    if (y >= m) dp[x+1][next_mask] += dp[x][mask];
    else{
        int my_mask = 1 << y;
        if(mask & my_mask){
            calc (x, y+1, mask, next_mask);
        }else{
            calc (x, y+1, mask, next_mask | my_mask);
            if (y+1 < m && ! (mask & my_mask) && ! (mask &
(my_mask << 1)))
                calc (x, y+2, mask, next_mask);
        }
    }
}

int main(){
    cin >> n >> m;
    dp.resize (n+1, vector<long long> (1<<m));
    dp[0][0] = 1;
    for (int x=0; x<n; ++x)
        for (int mask=0; mask<(1<<m); ++mask)
            calc (x, 0, mask, 0);
    cout << dp[n][0];
}

```

6.6 sos dp

```

// itera sobre los subconjuntos de todos los subconjuntos en
O(3^n)
for(int mask=0; mask<(1<<n); ++mask){
    for(int smask=mask; smask; smask=(smask-1)&mask){ //
decreasing order
        // for(int s=0;s=s-mask&mask;) // Increasing order
        ;
    }
}

```

```

}
}

// SOS DP en O(n * 2^n)
for(int mask = 0; mask < (1<<N); ++mask){
    dp[mask][-1] = A[mask]; //handle base case separately
(leaf states)
    for(int i = 0; i < N; ++i){
        if(mask & (1<<i)){
            dp[mask][i] = dp[mask][i-1] + dp[mask^(1<<i)][i-1];
        }else{
            dp[mask][i] = dp[mask][i-1];
        }
    }
    F[mask] = dp[mask][N-1]; // F es la acumulada en todas las
smask de mask
}

```

6.7 dp divide and conquer

```

// dp(i, j) = min dp(i-1, k-1) + C(k, j) for all k in [0, j]
// C(a, c) + C(b, d) <= C(a, d) + C(b, c) for all a <= b <= c
<= d
vector<pair<ll, ll>> c;
vl acum1, acum2;

ll cost(ll i, ll j){
    return c[j].first * (acum1[j+1] - acum1[i]) - (acum2[j+1]
- acum2[i]);
}
vector<ll> last, now;

void compute(int l, int r, int optl, int optr){
    if (l > r) return;
    int mid = (l + r) / 2;
    pair<ll, int> best = {cost(0, mid), -1};
    for(int k = max(1, optl); k < min(mid, optr) + 1; k++){
        best = min(best, {last[k - 1] + cost(k, mid), k});
        now[mid] = best.first;
    }

    compute(l, mid - 1, optl, best.second);
    compute(mid + 1, r, best.second, optr);
}

int main(){
    ios_base::sync_with_stdio(0); cin.tie(0);
    ll n, k, x, w;
    while(cin >> n >> k){
        c.clear();
        for(int i = 0; i < n; i++){
            cin >> x >> w;
            c.push_back({x, w});
        }
        acum1.clear(); acum2.clear();
        acum1.push_back(0); acum2.push_back(0);
    }
}

```



```

for(int i = 0; i < n; i++){
    acum1.push_back(c[i].second);
    acum2.push_back(c[i].first * c[i].second);
    acum1.back() += acum1[i];
    acum2.back() += acum2[i];
}
last.assign(n, INF);
now.resize(n);
for(int i = 0; i < k; i++) { compute(0, n - 1, 0, n -
1); swap(last, now); }

cout<< last[n-1]<<"\n";
}
}

```

```

indexed
assert(1 <= x1 && x1 <= x2 && x2 <= n);
assert(1 <= y1 && y1 <= y2 && y2 <= m);
x1--;
y1--;
int dx = x2 - x1, dy = y2 - y1;
return (1LL * (hs[x2][y2] - 1LL * hs[x2][y1] *
PWY[dy] % mod + mod) % mod -
1LL * (hs[x1][y2] - 1LL * hs[x1][y1] * PWY[dy] %
mod + mod) % mod * PWX[dx] % mod + mod) % mod;
}
int get_hash() {
    return get_hash(1, 1, n, m);
}
};

```

```

// para un texto s, recorrer las aristas por el arbol
void run(string &s){
    for(int p = 0, i = 0; i<(int)s.size(); ++i){
        int t = s[i]-'a';
        p = (N[p][t] == -1 ? 0 : N[p][t]);
        cnt[p]++;
    }
}
void dfs(int u = 0){
    vis[u] = true;
    for(int v : G[u]){
        if(v == -1) continue;
        if(!vis[v]) dfs(v);
        cnt[u] += cnt[v];
    }
    for(int v : val[u]){
        ans[v] = cnt[u];
    }
}
};

```

7 Strings

7.1 hashing2d

```

struct Hashing2D {
    vector<vector<int>> hs;
    vector<int> PWX, PWY;
    int n, m;
    static const int PX = 3731, PY = 2999, mod = 998244353;
    Hashing() {}
    Hashing(vector<string>& s) {
        n = (int)s.size(), m = (int)s[0].size();
        hs.assign(n + 1, vector<int>(m + 1, 0));
        PWX.assign(n + 1, 1);
        PWY.assign(m + 1, 1);
        for (int i = 0; i < n; i++) PWX[i + 1] = 1LL *
PWX[i] * PX % mod;
        for (int i = 0; i < m; i++) PWY[i + 1] = 1LL *
PWX[i] * PY % mod;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                hs[i + 1][j + 1] = s[i][j] - 'a' + 1;
            }
        }
        for (int i = 0; i <= n; i++) {
            for (int j = 0; j <= m; j++) {
                hs[i][j + 1] = (hs[i][j] + 1) + 1LL * hs[i]
[j] * PY % mod) % mod;
            }
        }
        for (int i = 0; i < n; i++) {
            for (int j = 0; j <= m; j++) {
                hs[i + 1][j] = (hs[i + 1][j] + 1LL * hs[i]
[j] * PX % mod) % mod;
            }
        }
    }
    int get_hash(int x1, int y1, int x2, int y2) { // 1-

```

7.2 aho corasick

```

struct AC
{
    ll c = 0, ec = 0, M, A, af = -1;
    vector<vl> N, G; vl L, E;
    vector<vl> val;
    vl vis, cnt;
    // L -> suffix link G -> anti L
    // E -> string finish
    AC (ll M, ll A) : M(M), A(A), N(M, vl(A, 0)), G(M,
vl(0)), E(M, 0), L(M, 0), val(M, vl(0)), vis(M, 0), cnt(M,
0){}
    ll add(string s, int id){ // return endpoint
        af++;
        ll p = 0;
        for (char l : s){
            int t = l - 'a';
            if (!N[p][t]) N[p][t] = ++c;
            p = N[p][t];
        }
        val[p].push_back(id);
        return p;
    }
    // construct aho corasick structure
    void init(){
        queue<int> q; q.push(0); L[0] = -1;
        while (!q.empty()){
            int p = q.front(); q.pop();
            for(int c = 0; c < A; c++){
                int u = N[p][c]; if (!u) continue;
                L[u] = L[p] == -1 ? 0 : N[L[p]][c],
q.push(u);
                G[L[u]].push_back(u);
            }
            if (p) for(int c = 0; c < A; c++) if (!N[p][c])
N[p][c] = N[L[p]][c];
        }
    }
};

```

7.3 hashing

```

struct RH {
    int binpow(int a, int b, int MOD) {
        int res = 1;
        while(b) {
            if(b & 1) res = (1LL * res * a) % MOD;
            a = (1LL * a * a) % MOD;
            b >>= 1;
        }
        return res;
    }
    // choose base B random to avoid hacks 33 37 41
    // randomize V(s[i])
    const int B = 119, M[2] = {999727999, 1070777777}, P[2]
= {325255434, 10018302};
    vector<int> H[2], I[2];
    RH(string &s) {
        int N = s.size();
        rep(k, 2) {
            H[k].resize(N + 1), I[k].resize(N + 1);
            H[k][0] = 0; ll b = 1;
            rep(i, N) {
                H[k][i + 1] = (H[k][i] + b * (s[i] - 'a' +
1)) % M[k];
                b = (b * B) % M[k];
            }
            I[k][N] = binpow(b, M[k] - 2, M[k]);
            for(int i = N - 1; i >= 0; i--) I[k][i] = (1LL *
I[k][i + 1] * B) % M[k];
        }
        ll get(int l, int r) // inclusive - exclusive
    }
};

```

```

ll h0 = (H[0][r] - H[0][l] + M[0]) % M[0];
h0 = (1LL * h0 * I[0][l]) % M[0];
ll h1 = (H[1][r] - H[1][l] + M[1]) % M[1];
h1 = (1LL * h1 * I[1][l]) % M[1];
return (h0 << 32) | h1;
}
};

```

7.4 kmp

```

vector<int> kmppre(string& t){ // r[i]: longest border of
t[0,i]
vector<int> r(t.size()+1);r[0]=-1;
int j=-1;
for(i,0,t.size()){
while(j>=0&&t[i]!=t[j])j=r[j];
r[i+1]=++j;
}
return r;
}
void kmp(string& s, string& t){ // find t in s
int j=0;vector<int> b=kmppre(t);
for(i,0,s.size()){
while(j>=0&&s[i]!=t[j])j=b[j];
if(++j==t.size())printf("Match at %d\n",i-j+1),j=b[j];
}
}

```

7.5 manacher

```

int n;
string s;

int main(){
vi d1(n); // odd sized palindromes
for (int i = 0, l = 0, r = -1; i < n; i++) {
int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
while (0 <= i - k && i + k < n && s[i - k] == s[i + k]) k++;
d1[i] = k--;
if (i + k > r) l = i - k, r = i + k;
}
vi d2(n); // even sized palindromes (center to the
right)
for (int i = 0, l = 0, r = -1; i < n; i++) {
int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
while (0 <= i - k - 1 && i + k < n && s[i - k - 1]
== s[i + k]) k++;
d2[i] = k--;
if (i + k > r) l = i - k - 1, r = i + k;
}
}

```

7.6 suffix array

```

#define rep(i, n) for(int i = 0; i < (int)n; ++i)
#define repx(i, a, b) for(int i = (int)a; i < (int)b; ++i)
#define invrep(i, b, a) for(int i = (int)b; i >= a; --i)
#define vl vector<ll>
struct SA {
int n; vl counts, rank, rank_, sa, sa_, lcp; // lcp is
optional
inline int gr(int i) { return i < n ? rank[i] : 0; }
void csort(int maxv, int k) {
counts.assign(maxv+1, 0);
repx(i,0,n) counts[gr(i+k)]++;
repx(i,1,maxv+1) counts[i] += counts[i-1];
invrep(i,n-1,0) sa_ [--counts[gr(sa[i]+k)]] = sa[i];
sa.swap(sa_);
}
void get_sa(vl& s) {
repx(i,0,n) sa[i] = i;
sort(sa.begin(), sa.end(), [&s](int i, int j))
{ return s[i] < s[j]; });
int r = rank[sa[0]] = 1;
repx(i,1,n) rank[sa[i]] = (s[sa[i]] != s[sa[i-1]]) ?
++r : r;
for (int h=1; h < n and r < n; h <= 1) {
csort(r, h); csort(r, 0); r = rank_[sa[0]] = 1;
repx(i,1,n) {
if (rank[sa[i]] != rank[sa[i-1]] or
gr(sa[i]+h) != gr(sa[i-1]+h)) ++r;
rank_[sa[i]] = r;
} rank.swap(rank_);
}
}
// LCP construction in O(N) using Kasai's algorithm
// reference: https://codeforces.com/blog/entry/12796?#
comment-175287
void get_lcp(vl& s) { // lcp is optional
lcp.assign(n, 0); int k = 0;
repx(i,0,n) {
int r = rank[i]-1;
if (r == n-1) { k = 0; continue; }
int j = sa[r+1];
while (i+k < n and j+k < n and s[i+k] == s[j+k]) k++;
lcp[r] = k;
if (k) k--;
}
}
SA(vl& s) {
n = s.size();
rank.resize(n); rank_.resize(n);
sa.resize(n); sa_.resize(n);
get_sa(s); get_lcp(s); // lcp is optional
}
};

```

```

int main() { // how to use
string test; cin >> test;
vl s;
for (char c : test) s.push_back(c);
SA sa(s);
for (int i : sa.sa) cout << i << ": " << test.substr(i)
<< '\n';
repx(i,0,s.size()) {
printf("LCP between %d and %d is %d\n", i, i+1,
sa.lcp[i]);
}
}

```

7.7 trie

```

struct Trie{
static const int MAX = 1e6;
int N[MAX][26] = {0}, S[MAX] = {0}, c = 0;
void add(string s, int a = 1){
int p = 0; S[p] += a;
for (char l : s){
int t = l - 'a';
if (!N[p][t]) N[p][t] = ++c;
S[p] = N[p][t]; ++a;
}
}
};

struct TrieXOR{
static const int MAX = 1e6;
int N[MAX][2] = {0}, S[MAX] = {0}, c = 0;
void add(int x, int a = 1){
int p = 0; S[p] += a;
rep(i, 31){
int t = (x >> (30 - i)) & 1;
if (!N[p][t]) N[p][t] = ++c;
S[p] = N[p][t]; ++a;
}
}
int get(int x){
if (!S[0]) return -1;
int p = 0; rep(i, 31){
int t = ((x >> (30 - i)) & 1) ^ 1;
if (!N[p][t] || !S[N[p][t]]) t ^= 1;
p = N[p][t]; if (t) x ^= (1 << (30 - i));
}
return x;
}
};

struct Trie {
vector<vector<int>> g;
vector<int> count;
int vocab;

```

```

Trie(int vocab, int maxdepth = 10000) : vocab(vocab) {
    g.reserve(maxdepth);
    g.emplace_back(vocab, -1);
    count.reserve(maxdepth);
    count.push_back(0);
}

int move_to(int u, int c) {
    assert (0 <= c and c < vocab);
    int& v = g[u][c];
    if (v == -1) {
        v = g.size();
        g.emplace_back(vocab, -1);
        count.push_back(0);
    }
    count[v]++;
    return v;
}

void insert(const string& s, char ref = 'a') { //
insert string
    int u = 0; for (char c : s) u = move_to(u, c - ref);
}

void insert(vector<int>& s) { // insert vector<int>
    int u = 0; for (int c : s) u = move_to(u, c);
}

db query(const string& s, char ref = 'a'){
    int u = 0;
    db cost = 0;
    for (char c : s){
        ll co = 0;
        for(auto it : g[u]) if(it != -1)co++;

        ll nex = move_to(u, c - ref);
        if(u == 0 || co > 1 || count[u] != count[nex]) cost+
+;
        u = nex;
    }
    return cost;
}

ll dfs(int u, int depht){
    ll ans = INF;
    if(count1[u] == 1 && count2[u] == 1)ans = depht;
    for(int i = 0; i < 26; i++){
        if(g[u][i] != -1) ans = min(ans, dfs(g[u][i],
depht + 1));
    }
    return ans;
}

int size() { return g.size(); }
};

```

7.8 suffix automaton

```

struct State {int len, link; map<char,int> next; };
State st[2*MAXN]; int sz, last; // clear next!!
void sa_init(){ last=st[0].len=0; sz=1; st[0].link=-1; }

```

```

void sa_extend(char c){// total build O(n log alphabet_size)
    int k = sz++, p; st[k].len = st[last].len + 1;
    for(p=last; p!=-1 && !st[p].next[c]; p=st[p].link)
        st[p].next[c] = k;
    if(p == -1) st[k].link = 0;
    else {
        int q = st[p].next[c];
        if(st[p].len + 1 == st[q].len) st[k].link = q;
        else {
            int w = sz++; st[w].len = st[p].len + 1;
            st[w].next=st[q].next; st[w].link=st[q].link;
            for(; p!=-1 && st[p].next[c]==q; p=st[p].link)
                st[p].next[c] = w;
            st[q].link=st[k].link = w;
        }
    }
    last = k;
} // # states <= 2n-1 && transitions <= 3n-4 (for n > 2)
// Follow link from `last` to 0, nodes on path are terminal
// # matches = # paths from state to a terminal node
// # substrings = # paths from 0 to any node
// # substrings = sum of (len - len(link)) for all nodes

```

7.9 palindromic tree

```

struct Node { // (*) = Optional
    int len; // length of substring
    int to[26]; // insertion edge for all characters a-z
    int link; // maximun palindromic suffix
    int i; // (*) start index of current Node
    int cnt; // (*) # of occurrences of this substring
    Node(int len, int link=0, int i=0, int cnt=1): len(len),
link(link), i(i), cnt(cnt) {memset(to, 0, sizeof(to));}
}; struct EerTree { // Palindromic Tree
    vector<Node> t; // tree (max size of tree is n+2)
    int last; // current node
    EerTree(string &s) : last(0) {
        t.emplace_back(-1); t.emplace_back(0); // root 1 & 2
        rep(i, s.size()) add(i, s); // construct tree
        for(int i = t.size()-1; i > 1; i--)
            t[t[i].link].cnt += t[i].cnt;
    }

    void add(int i, string &s){ // vangrind warning:
        int p=last, c=s[i]-'a'; // i-t[p].len-1 = -1
        while(s[i-t[p].len-1] != s[i]) p = t[p].link;
        if(t[p].to[c]){ last = t[p].to[c]; t[last].cnt++; }
        else{
            int q = t[p].link;
            while(s[i-t[q].len-1] != s[i]) q = t[q].link;
            q = max(1, t[q].to[c]);
            last = t[p].to[c] = t.size();
            t.emplace_back(t[p].len + 2, q, i-t[p].len-1);
        }
    }
};

```

```

void main(){
    string s = "abcbab"; EerTree pt(s); // build EerTree
    repx(i, 2, pt.t.size()){// list all distinct palindromes
        repx(j, pt.t[i].i, pt.t[i].i+pt.t[i].len)cout << s[j];
        cout << " " << pt.t[i].cnt << endl;
    }
}

```

7.10 rolling hash

```

struct RollingHash {
    static const ll BASE = 31;
    static const ll MOD1 = 1e9 + 9, MOD2 = 1e9 + 7, MOD3 =
(119LL << 23) | 1;
    const vector<ll> MODS = {MOD1, MOD2, MOD3};

    vector<vector<ll>> prefixes, powers;
    int n;

    RollingHash(const string &text) {
        n = text.size();
        prefixes.assign(3, vector<ll>(n + 2));
        powers.assign(3, vector<ll>(n + 2, 1));

        for (int j = 0; j < 3; j++)
            for (int i = 1; i <= n + 1; i++)
                powers[j][i] = (powers[j][i - 1] * BASE) % MODS[j];

        for (int j = 0; j < 3; j++) {
            for (int i = 0; i < n; i++)
                prefixes[j][i + 1] = (prefixes[j][i] * BASE +
text[i]) % MODS[j];

            prefixes[j][n + 1] = (prefixes[j][n] * BASE) %
MODS[j];
        }

        tuple<int, int, int> query(int left, int right) {
            vector<int> result(3);

            for (int i = 0; i < 3; i++) {
                ll hash = (prefixes[i][right + 1] -
prefixes[i][left] * powers[i][right - left
+ 1] + MODS[i]) %
MODS[i];

                if (hash < 0)
                    hash += MODS[i];

                result[i] = hash;
            }

            return {result[0], result[1], result[2]};
        }
    }
}

```

```
}  
};
```

Distribución	Densidad de Probabilidad	Θ_X	Parámetros	Esperanza y Varianza
Binomial	$\binom{n}{x} p^x (1-p)^{n-x}$	$x = 0, \dots, n$	n, p	$\mu_X = n p$ $\sigma_X^2 = n p (1-p)$ $M(t) = [p e^t + (1-p)]^n, \quad t \in \mathbb{R}$
Geométrica	$p (1-p)^{x-1}$	$x = 1, 2, \dots$	p	$\mu_X = 1/p$ $\sigma_X^2 = (1-p)/p^2$ $M(t) = p e^t / [1 - (1-p) e^t], \quad t < -\ln(1-p)$
Binomial-Negativa	$\binom{x-1}{r-1} p^r (1-p)^{x-r}$	$x = r, r+1, \dots$	r, p	$\mu_X = r/p$ $\sigma_X^2 = r(1-p)/p^2$ $M(t) = \{p e^t / [1 - (1-p) e^t]\}^r, \quad t < -\ln(1-p)$
Poisson	$\frac{(\nu t)^x e^{-\nu t}}{x!}$	$x = 0, 1, \dots$	ν	$\mu_X = \nu t$ $\sigma_X^2 = \nu t$ $M(t) = \exp[\lambda (e^t - 1)], \quad t \in \mathbb{R}$
Exponencial	$\nu e^{-\nu x}$	$x \geq 0$	ν	$\mu_X = 1/\nu$ $\sigma_X^2 = 1/\nu^2$ $M(t) = \nu / (\nu - t), \quad t < \nu$
Gamma	$\frac{\nu^k}{\Gamma(k)} x^{k-1} e^{-\nu x}$	$x \geq 0$	k, ν	$\mu_X = k/\nu$ $\sigma_X^2 = k/\nu^2$ $M(t) = [\nu / (\nu - t)]^k, \quad t < \nu$
Normal	$\frac{1}{\sqrt{2\pi}\sigma} \exp\left[-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2\right]$	$-\infty < x < \infty$	μ, σ	$\mu_X = \mu$ $\sigma_X^2 = \sigma^2$ $M(t) = \exp(\mu t + \sigma^2 t^2 / 2), \quad t \in \mathbb{R}$
Log-Normal	$\frac{1}{\sqrt{2\pi}(\zeta x)} \exp\left[-\frac{1}{2}\left(\frac{\ln x - \lambda}{\zeta}\right)^2\right]$	$x \geq 0$	λ, ζ	$\mu_X = \exp\left(\lambda + \frac{1}{2}\zeta^2\right)$ $\sigma_X^2 = \mu_X^2 (e^{\zeta^2} - 1)$ $E(X^r) = e^{r\lambda} M_Z(r\zeta), \text{ con } Z \sim \text{Normal}(0,1)$
Uniforme	$\frac{1}{(b-a)}$	$a \leq x \leq b$	a, b	$\mu_X = (a+b)/2$ $\sigma_X^2 = (b-a)^2/12$ $M(t) = [e^t b - e^t a] / [t(b-a)], \quad t \in \mathbb{R}$
Beta	$\frac{1}{B(q, r)} \frac{(x-a)^{q-1} (b-x)^{r-1}}{(b-a)^{q+r-1}}$	$a \leq x \leq b$	q, r	$\mu_X = a + \frac{q}{q+r} (b-a)$ $\sigma_X^2 = \frac{q r (b-a)^2}{(q+r)^2 (q+r+1)}$
Hipergeométrica	$\frac{\binom{m}{x} \binom{N-m}{n-x}}{\binom{N}{n}}$	$\max\{0, n+m-N\} \leq x \leq \min\{n, m\}$	N, m, n	$\mu_X = n \frac{m}{N}$ $\sigma_X^2 = \left(\frac{N-n}{N-1}\right) n \frac{m}{N} \left(1 - \frac{m}{N}\right)$